Dossier de projet — InnovShop

Oct 18, 2026 · @Hermann

InnovShop — Le futur, livré chez vous. Boutique en ligne d'objets futuristes développée avec Symfony 7.4. Dossier présenté par **Hermann MBALA** pour le titre professionnel **Développeur Web et Web Mobile** (niveau 5, RNCP 37674), formation suivie à Global Digital University. Site en ligne : https://hermann.alwaysdata.net/innovshop/ — Code source : https://github.com/LettreH/innovshop

1. Compétences mises en œuvre

InnovShop couvre les **8 compétences** du titre Développeur Web et Web Mobile : les 4 du bloc front-end et les 4 du bloc back-end.

Bloc	Compétence (RNCP 37674)	Où elle apparaît dans InnovShop
Front-end	Installer et configurer son environnement de travail en fonction du projet web ou web mobile	Chromebook sous Linux, PHP 8.2, Composer, Symfony CLI, MariaDB, Git et GitHub (partie 4)
Front-end	Maquetter des interfaces utilisateur web ou web mobile	Organisation des pages et parcours d'achat, mise en page « mobile first » avec la grille Bootstrap (partie 6)
Front-end	Réaliser des interfaces utilisateur statiques web ou web mobile	Templates Twig, héritage de gabarit, morceaux réutilisables, Bootstrap 5 responsive, accessibilité (partie 6)
Front-end	Développer la partie dynamique des interfaces utilisateur web ou web mobile	Panier en AJAX avec `fetch`, réponses JSON, messages d'erreur, jeton CSRF (partie 6)
Back-end	Mettre en place une base de données relationnelle	7 entités Doctrine, relations, migrations, index (partie 5)
Back-end	Développer des composants d'accès aux données SQL et NoSQL	Repositories, QueryBuilder, requêtes paramétrées, correction du problème N+1 (partie 7)
Back-end	Développer des composants métier côté serveur	Panier en session, validation de commande, emails, facture PDF, Voters, tests PHPUnit (parties 7 et 8)

Bloc	Compétence (RNCP 37674)	Où elle apparaît dans InnovShop
Back-end	Documenter le déploiement d'une application dynamique web ou web mobile	Mise en ligne sur Alwaysdata et procédure écrite dans le README (partie 10)

Le NoSQL n'est pas utilisé dans InnovShop : les données (clients, commandes, produits) sont très liées entre elles, une base relationnelle est donc le bon choix.

2. Résumé du projet

InnovShop est une **boutique en ligne complète** d'objets futuristes, réalisée pour une cliente fictive, Madame Leroy. Elle est développée avec le framework **Symfony 7.4** et elle est **en ligne** sur l'hébergeur Alwaysdata.

- **Côté client :** un catalogue avec recherche et filtre, des fiches produits avec options (taille, couleur…), un panier qui se met à jour sans recharger la page, une commande en 3 étapes, un email de confirmation, et un espace client avec le suivi des commandes et la facture en PDF.
- **Côté administrateur :** un back-office pour gérer les catégories, les produits, les commandes et leur statut, et les clients.
- **Qualité :** 17 tests automatisés, des protections contre les attaques courantes, et un score **Lighthouse de 100** dans les 4 catégories (performances, accessibilité, bonnes pratiques, référencement).

J'ai réalisé ce projet seul, en 10 séquences, pendant ma formation à Global Digital University. Les produits sont imaginaires et aucune vente réelle n'a lieu.

3. Contexte et expression des besoins

3.1 La cliente et son besoin

Madame Leroy lance une start-up qui vend des **objets technologiques du futur**. Elle veut une plateforme e-commerce complète : une vitrine pour présenter ses produits, un parcours d'achat simple, un espace pour ses clients, et un outil pour gérer sa boutique sans développeur.

3.2 Le cahier des charges

Partie	Ce qui est demandé
Front office	Accueil, catalogue, fiche produit, panier, inscription et connexion, commande, espace client
Back office	Gestion des produits, des commandes et des clients
Technique	Framework **Symfony** avec **Twig** pour l'affichage, validation des données, au moins trois bonnes pratiques Symfony, site déployé sur un serveur distant

J'ai relevé deux points flous dans le cahier des charges, et je les ai notés dans le backlog :

- il parlait de « bonnes pratiques Laravel », alors que le framework imposé est Symfony : j'ai appliqué les bonnes pratiques **Symfony** ;
- l'espace client évoquait un « suivi des objectifs d'épargne », qui ne correspond pas à une boutique : je l'ai remplacé par le **suivi des commandes**.

3.3 Les contraintes et les livrables

- **Contraintes :** Symfony et Twig imposés ; données validées côté serveur ; site sécurisé ; site utilisable sur ordinateur, tablette et téléphone. Mon mentor m'a autorisé à utiliser des **produits imaginaires**.
- **Livrables :** le backlog, le diagramme de classes, une capture du tableau Kanban, le dépôt GitHub documenté et le site en ligne.

3.4 Le backlog : 42 user stories

J'ai traduit le besoin en **user stories**, sur le modèle : « *En tant que [rôle], je veux [fonctionnalité], afin de [objectif].* » Chacune a une priorité (haute, moyenne, basse) et une complexité.

Partie	Nombre	Exemple
Front office	26	US09 — En tant que visiteur, je veux ajouter un produit à mon panier, afin de préparer ma commande.
Back office	12	US34 — En tant qu'administrateur, je veux mettre à jour le statut d'une commande, afin d'informer le client de l'avancement.

Partie	Nombre	Exemple
Technique	4	US40 — En tant que développeur, je veux déployer l'application sur un serveur distant, afin de rendre la boutique accessible en ligne.

Bilan : 41 user stories sur 42 sont terminées. La US37 (supprimer un compte client) n'a pas été retenue volontairement : supprimer un client effacerait aussi l'historique de ses commandes. Deux user stories (US41 facture PDF, US42 optimisation) ont été ajoutées en cours de route : le backlog a évolué avec le projet.

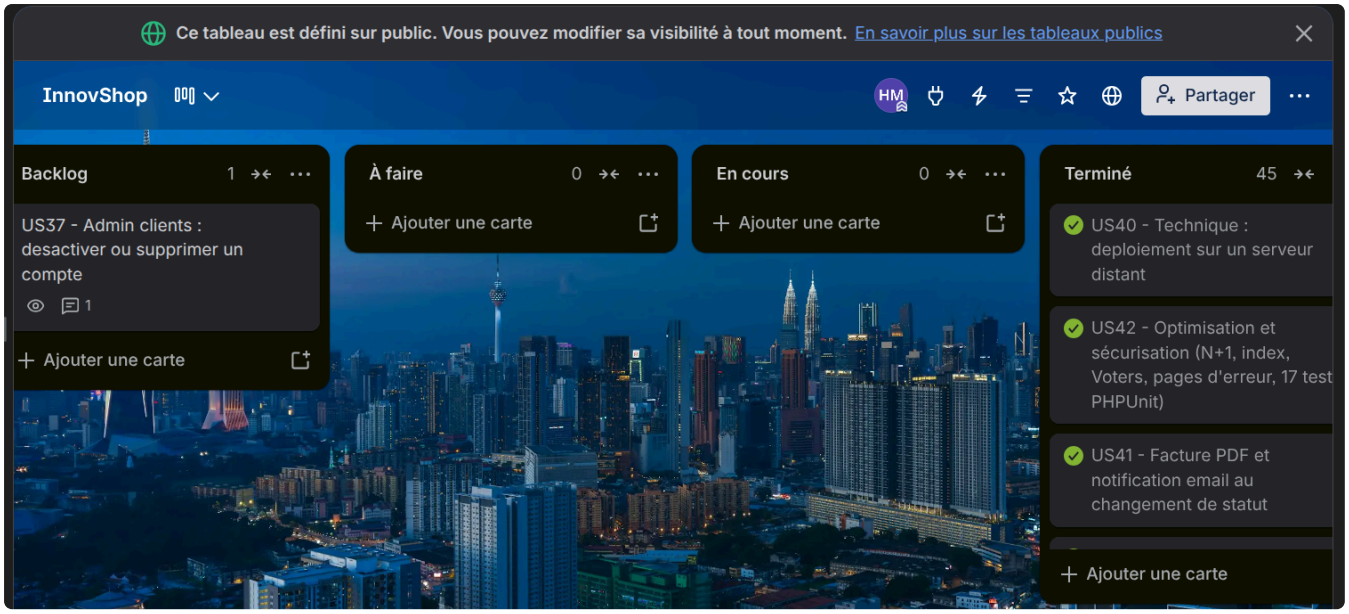
3.5 L'organisation du travail

J'ai suivi l'avancement sur un **tableau Kanban Trello** à 4 colonnes : Backlog, À faire, En cours, Terminé. Je déplaçais chaque carte au fil du travail, et je faisais un **commit Git** à la fin de chaque séquence.

Le projet s'est déroulé en 10 séquences :

Séquence	Contenu
1 et 2	Analyse du besoin, backlog, installation de Symfony
3	Entités Doctrine et première migration
4	Authentification, inscription et rôles
5	Back-office EasyAdmin
6	Accueil, catalogue, recherche, fiche produit, responsive
7	Panier en session et AJAX, commande en 3 étapes, email de confirmation
8	Espace client, facture PDF, email au changement de statut
9	Optimisation, sécurité, pages d'erreur, tests
10	Déploiement et documentation

Tableau Trello public : https://trello.com/b/CTkdf2jz/innovshop



Le tableau Trello en fin de projet : toutes les cartes sont dans « Terminé », sauf la US37 laissée volontairement dans le Backlog avec un commentaire qui explique pourquoi.

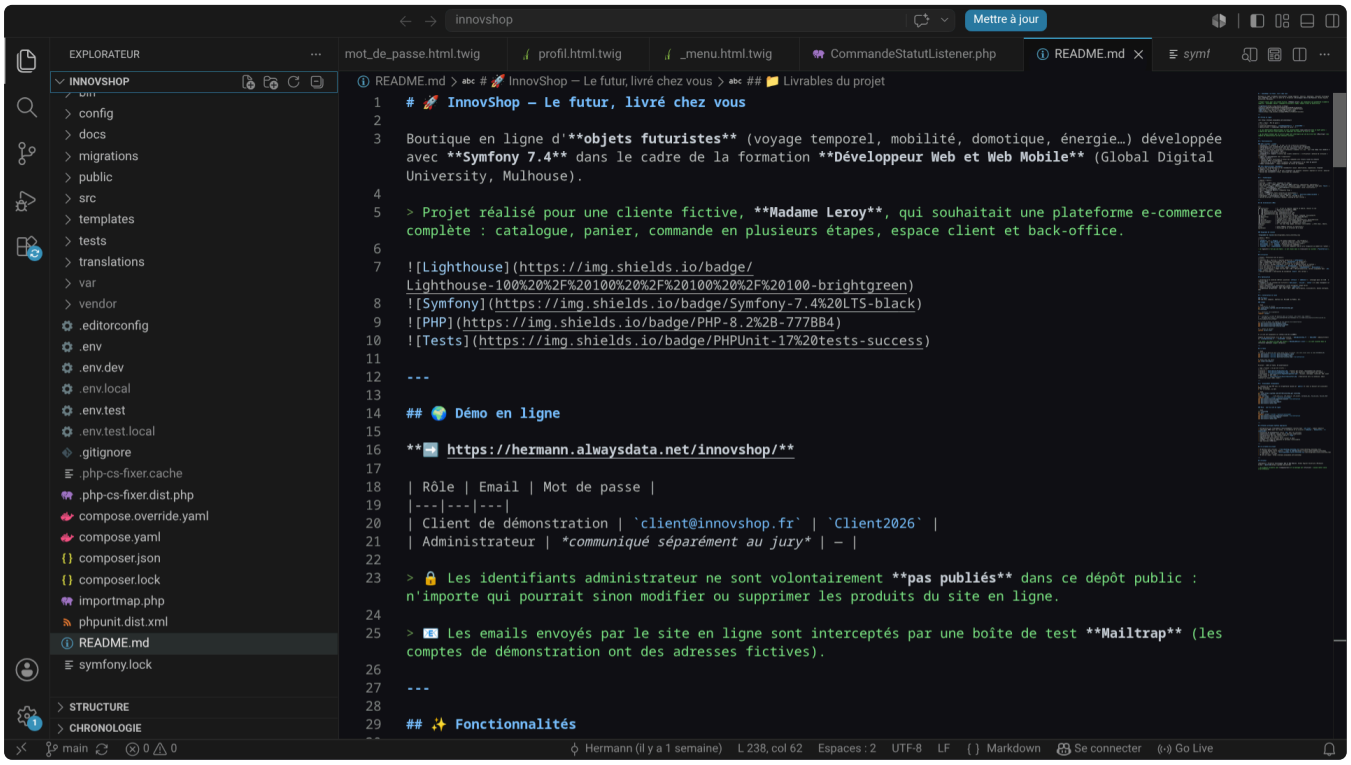
4. Environnement technique

4.1 Mon poste de travail

Je développe sur un **Chromebook Acer 314** avec l'environnement **Linux (Debian 12)** activé. Tous mes outils s'installent et se lancent dans le terminal.

Outil	Rôle
Visual Studio Code	Éditeur de code
PHP 8.2 et Composer	Langage du serveur et gestionnaire de bibliothèques PHP
Symfony CLI	Créer le projet, lancer le serveur local (`symfony server:start`), vérifier la configuration
MariaDB	Serveur de base de données en local
Git et GitHub	Historique des versions et sauvegarde en ligne du code
Chrome DevTools	Mode responsive, console, onglet Réseau, audit Lighthouse
Trello	Suivi des tâches en Kanban

La connexion à la base de données est dans un fichier **`.env.local`**, que le `.gitignore` empêche d'envoyer sur GitHub : mes mots de passe restent privés.



Le projet ouvert dans Visual Studio Code : l'arborescence Symfony (`config`, `migrations`, `src`, `templates`, `tests`…).

4.2 Les technologies et pourquoi je les ai choisies

Domaine	Technologie	Pourquoi ce choix
Framework	**Symfony 7.4 LTS**	Imposé par le cahier des charges. J'ai pris la version **LTS** (maintenue longtemps) car Symfony 8 demande PHP 8.4 et j'ai PHP 8.2.
Base de données	**MariaDB + Doctrine ORM**	Données très liées (clients, commandes, produits) : une base relationnelle convient. Doctrine me fait manipuler des objets PHP au lieu d'écrire du SQL.
Affichage	**Twig**	Imposé. Il échappe automatiquement le texte, ce qui protège contre le XSS.
Mise en page	**Bootstrap 5.3**	Grille responsive prête à l'emploi, composants accessibles.
Interactivité	**JavaScript** (`fetch`)	Mettre à jour le panier sans recharger la page.

Domaine	Technologie	Pourquoi ce choix
Back-office	**EasyAdmin 5**	Génère un back-office complet et sécurisé en peu de code.
Emails	**Symfony Mailer**	Emails de confirmation écrits avec des templates Twig.
PDF	**Dompdf**	Transforme un template Twig en facture PDF.
Assets	**AssetMapper** + **MinifyBundle**	Pas besoin de Node.js en production ; fichiers CSS et JS versionnés et minifiés.
Tests et qualité	**PHPUnit**, **PHP CS Fixer**	Tests automatiques ; code conforme à la norme **PSR-12**.
Hébergement	**Alwaysdata**	Hébergeur français avec offre gratuite, PHP 8.4, MariaDB et accès SSH.

4.3 Les bonnes pratiques Symfony appliquées

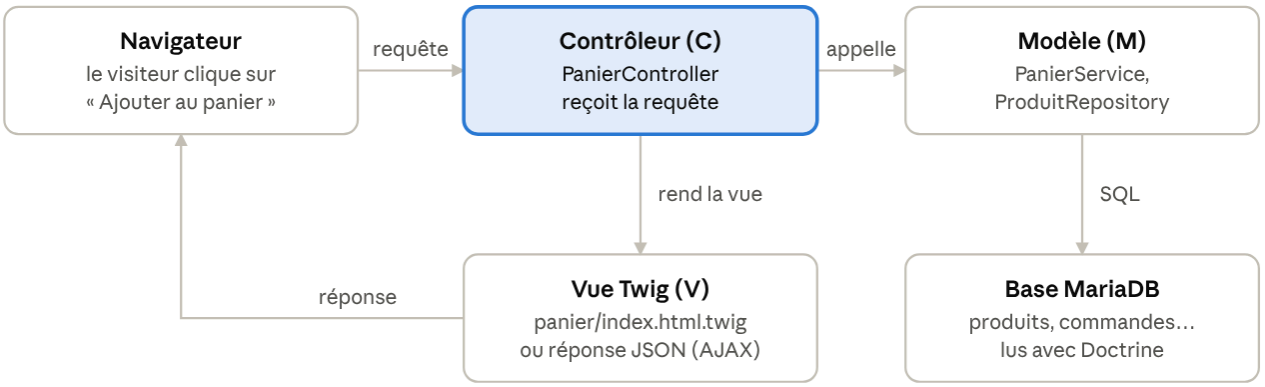
- Configuration par **variables d'environnement** ; secrets uniquement dans `.env.local`.
- **Attributs PHP** pour les routes, la validation et la sécurité (`#[Route]`, `#[Assert\…]`, `#[IsGranted]`).
- **Injection de dépendances** : je ne crée jamais un service avec `new`, Symfony me le fournit.
- **Controllers légers** : la logique métier est dans des **services**.
- **Formulaires** dans des classes dédiées (`AdresseType`, `ProfilType`…).
- **Voters** pour les règles d'accès, **migrations** Doctrine pour faire évoluer la base.

5. Conception

5.1 L'architecture MVC

Symfony organise le code en **MVC** : le **Modèle** (les données), la **Vue** (ce que voit l'utilisateur) et le **Contrôleur** (qui fait le lien). Quand un visiteur ouvre une page, sa requête suit toujours le même chemin.

Chaque requête suit le même chemin : contrôleur, modèle, vue



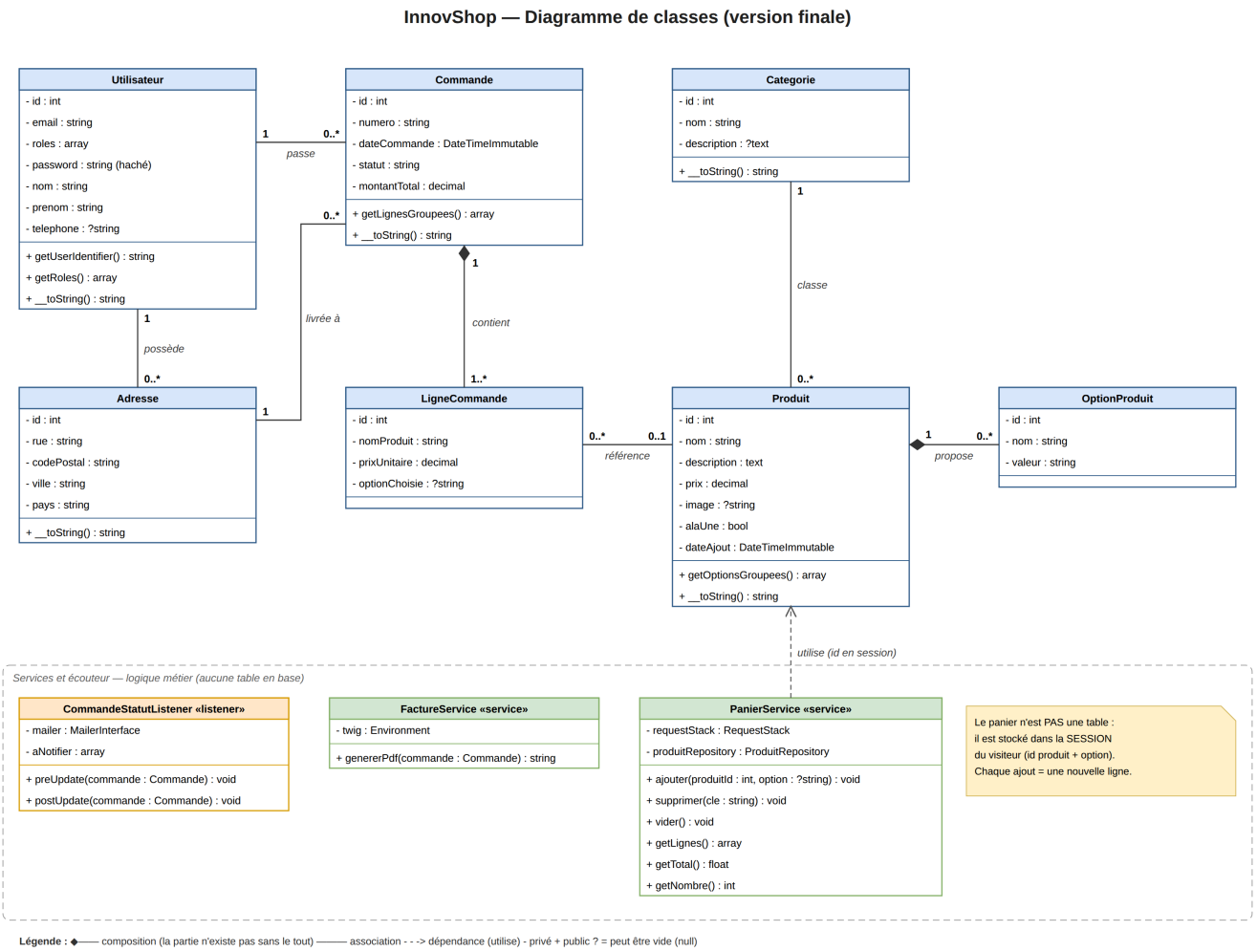
Le chemin d'une requête dans InnovShop · exemple de l'ajout au panier

Exemple : le clic sur « Ajouter au panier » arrive au `PanierController`, qui appelle le `PanierService` ; la réponse repart vers le navigateur, en HTML ou en JSON quand la demande vient d'AJAX.

Dossier	Rôle dans InnovShop
`src/Controller/`	**C** — reçoit la requête, appelle les services et les repositories, choisit la vue
`src/Entity/`	**M** — les objets métier : `Produit`, `Commande`, `Utilisateur`…
`src/Repository/`	**M** — les requêtes vers la base (QueryBuilder)
`src/Service/`	Logique métier réutilisable : `PanierService`, `FactureService`
`src/Form/`	Formulaires et règles de validation
`src/Security/Voter/`	Règles d'accès fines : `CommandeVoter`, `AdresseVoter`
`src/EventListener/`	Email automatique au changement de statut d'une commande
`templates/`	**V** — les vues Twig : pages, morceaux réutilisables, emails, facture

5.2 Le diagramme de classes

Le projet a **7 entités**. Le panier n'est pas une table : il est rangé dans la **session** du visiteur.



Le diagramme de classes final : les 7 entités en bleu, et en bas les services et l'écouteur qui portent la logique métier sans table en base.

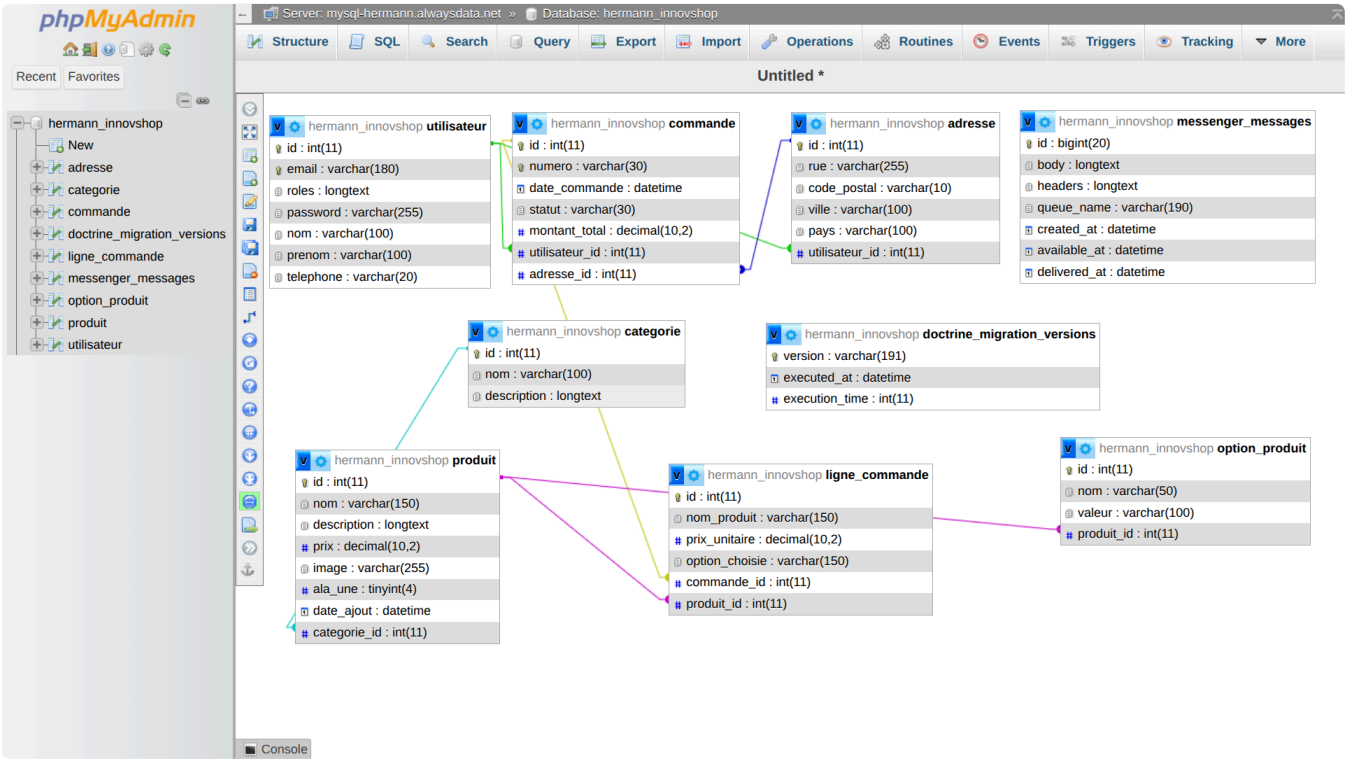
Relation	Sens
`Categorie` 1 — 0..* `Produit`	Une catégorie classe plusieurs produits
`Produit` 1 ◆— 0..* `OptionProduit`	Un produit propose des options (taille, couleur…). Composition : une option n'existe pas sans son produit.
`Utilisateur` 1 — 0..* `Adresse`	Un client possède ses adresses de livraison
`Utilisateur` 1 — 0..* `Commande`	Un client passe des commandes
`Adresse` 1 — 0..* `Commande`	Une commande est livrée à une adresse
`Commande` 1 ◆— 1..* `LigneCommande`	Une commande contient au moins une ligne
`LigneCommande` 0..* — 0..1 `Produit`	Une ligne référence le produit acheté

5.3 La base de données

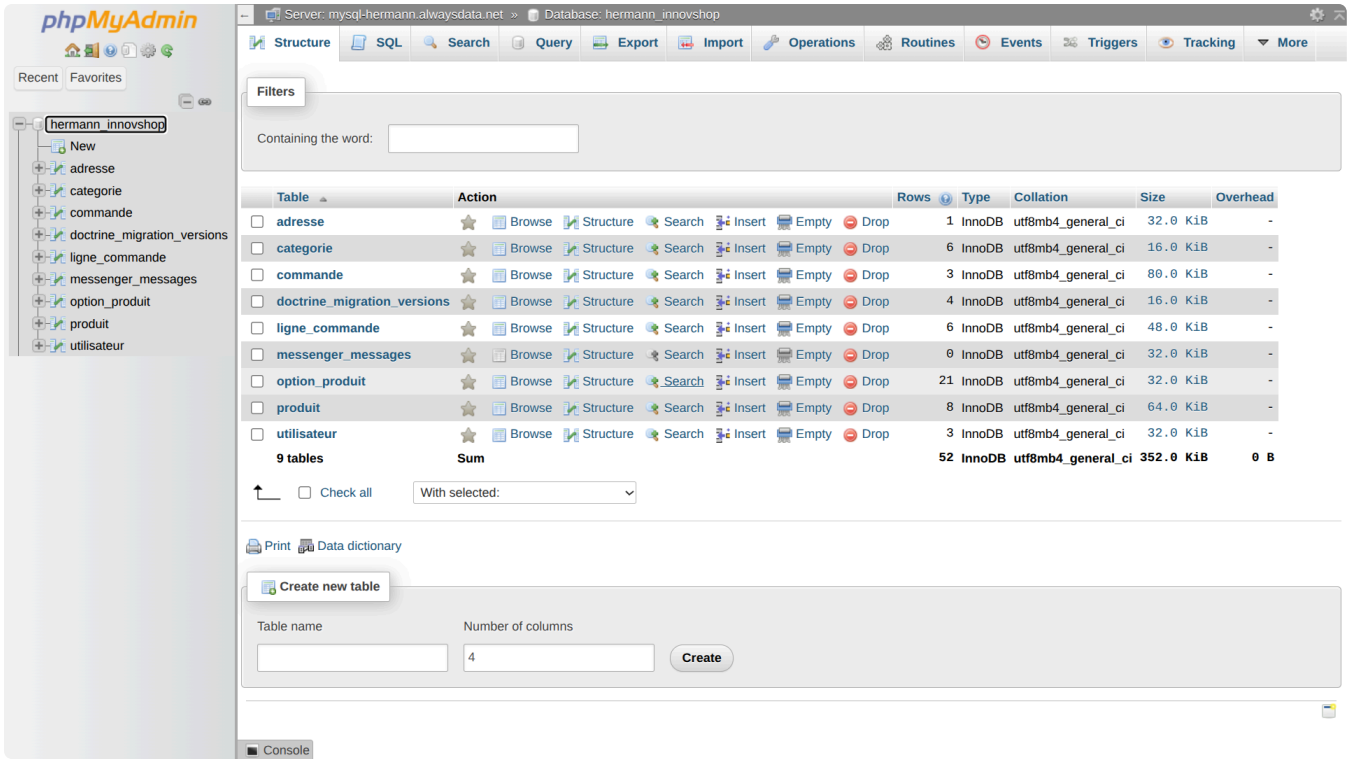
Les tables sont créées par les **migrations Doctrine** à partir des entités. Le projet en compte 4 : chaque changement de structure est un fichier, rejoué dans le même ordre sur mon ordinateur et sur le serveur.

J'ai fait plusieurs **choix de conception** importants :

- **Le nom et le prix sont copiés dans la ligne de commande** (`nomProduit`, `prixUnitaire`). Si l'administrateur change un prix plus tard, les anciennes commandes et leurs factures ne changent pas.
- **Un produit supprimé ne casse pas l'historique** : la clé étrangère de la ligne vers le produit est en `ON DELETE SET NULL`. La ligne garde son nom et son prix.
- **Le numéro de commande est unique** (contrainte `unique` en base), au format `CMD-20261017-89DAD4`.
- **Des index** sur les colonnes utilisées pour trier et filtrer : `date_ajout` et `ala_une` (accueil), `statut` (commandes).
- **Le mot de passe** est stocké **haché** (bcrypt), jamais en clair.



La base en ligne vue dans le Concepteur de phpMyAdmin : les tables et leurs clés étrangères.



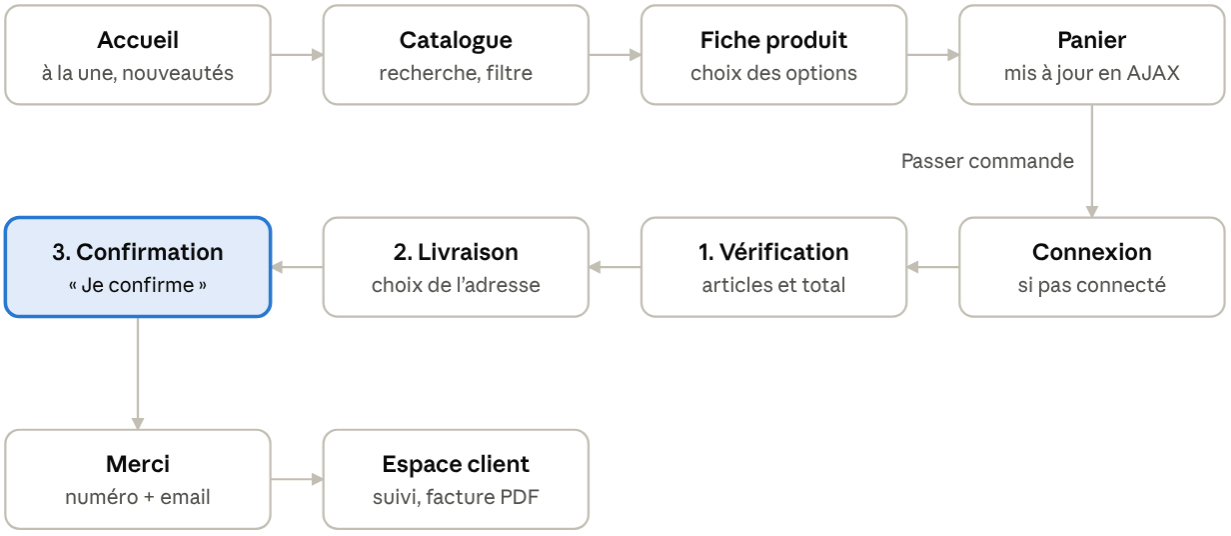
Les 9 tables, en InnoDB et en `utf8mb4`. `doctrine_migration_versions` et `messenger_messages` sont créées par Symfony.

6. Réalisations front-end

6.1 L'enchaînement des pages

Avant de coder les pages, j'ai défini le **parcours d'achat** : par quelles pages passe un visiteur, de l'accueil jusqu'à sa commande. La commande est réservée aux clients connectés : un visiteur est envoyé vers la page de connexion, puis revient à sa commande.

De l'accueil à la commande, en 10 écrans



Le parcours d'achat · 10 écrans, la commande en 3 étapes

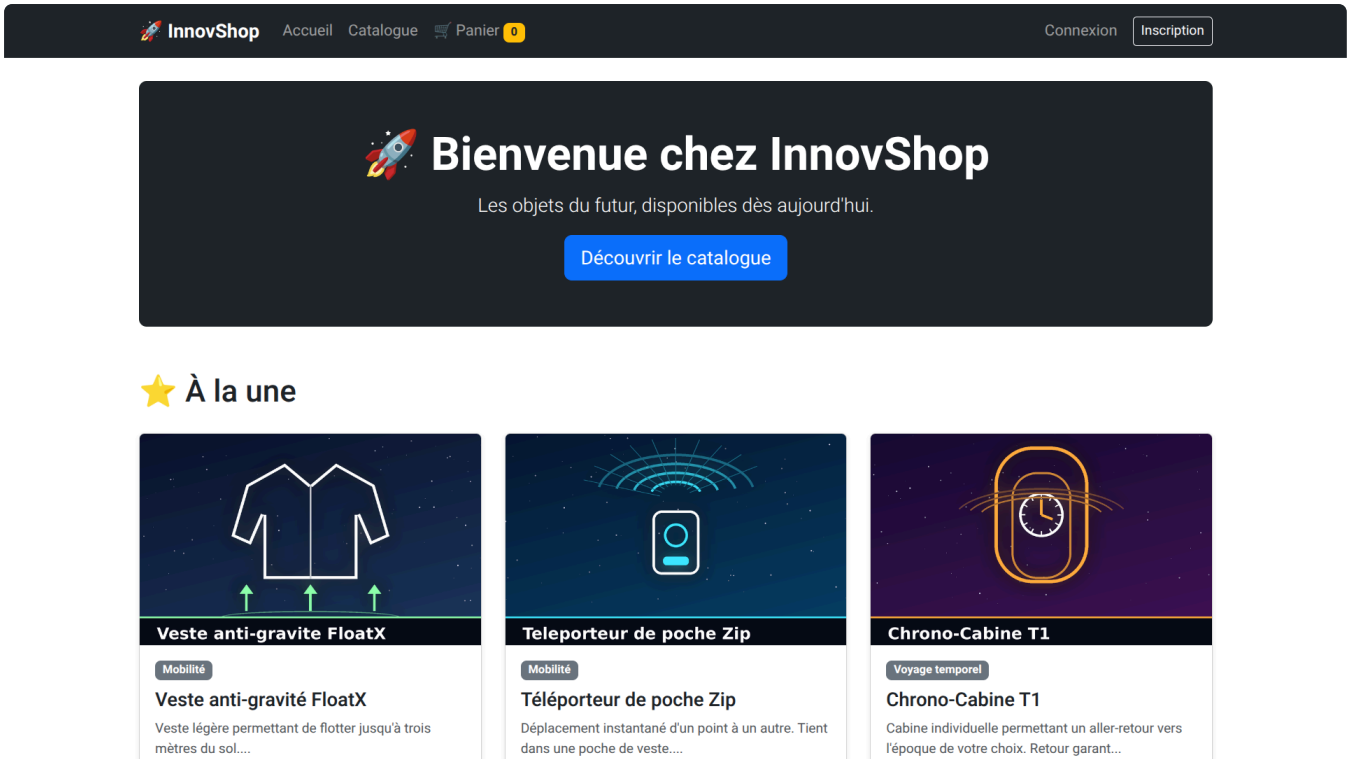
Le client voit toujours où il en est : la barre des 3 étapes (`_etapes.html.twig`) est affichée en haut de chaque écran de la commande.

6.2 Des pages construites avec Twig

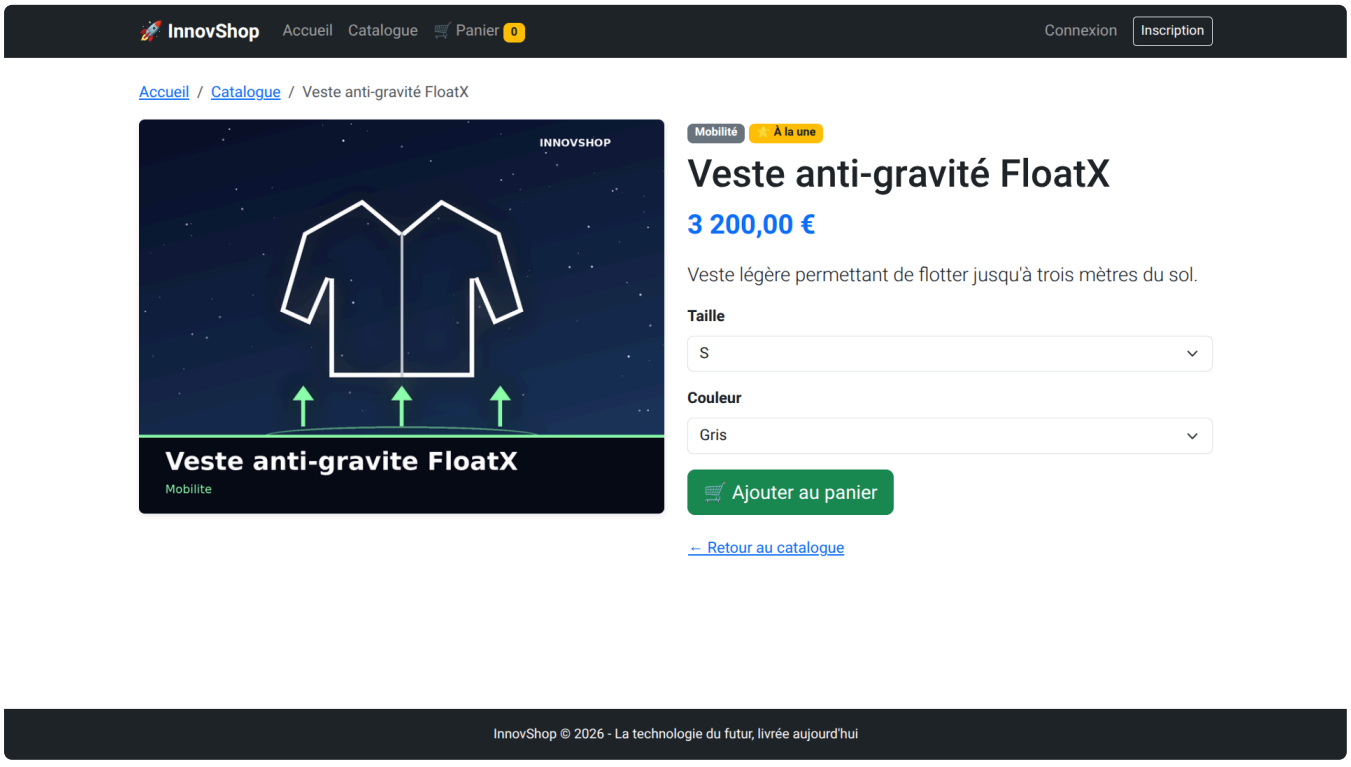
Toutes les pages héritent d'un **gabarit commun**, `base.html.twig` , qui contient le menu, le pied de page et les liens vers Bootstrap. Chaque page ne remplit que son bloc `body` . Les éléments qui se répètent sont des **morceaux réutilisables** (fichiers qui commencent par `_`) :

Morceau	Où il sert
`produit/_carte.html.twig`	La carte d'un produit, sur l'accueil et dans le catalogue
`commande/_etapes.html.twig`	La barre des 3 étapes de la commande
`commande/_recapitulatif.html.twig`	Le tableau récapitulatif du panier pendant la commande
`compte/_menu.html.twig`	Le menu de l'espace client
`compte/_suivi.html.twig`	Le suivi visuel du statut d'une commande

Tous les formulaires utilisent le thème **Bootstrap 5** de Symfony (`form_themes`) : les champs ont le bon style et les erreurs s'affichent en rouge sous le champ concerné.



La page d'accueil : la bannière, puis les produits « à la une » affichés avec le morceau `_carte.html.twig`.



La fiche produit : le fil d'Ariane, le prix, la description et le choix des options (taille, couleur).

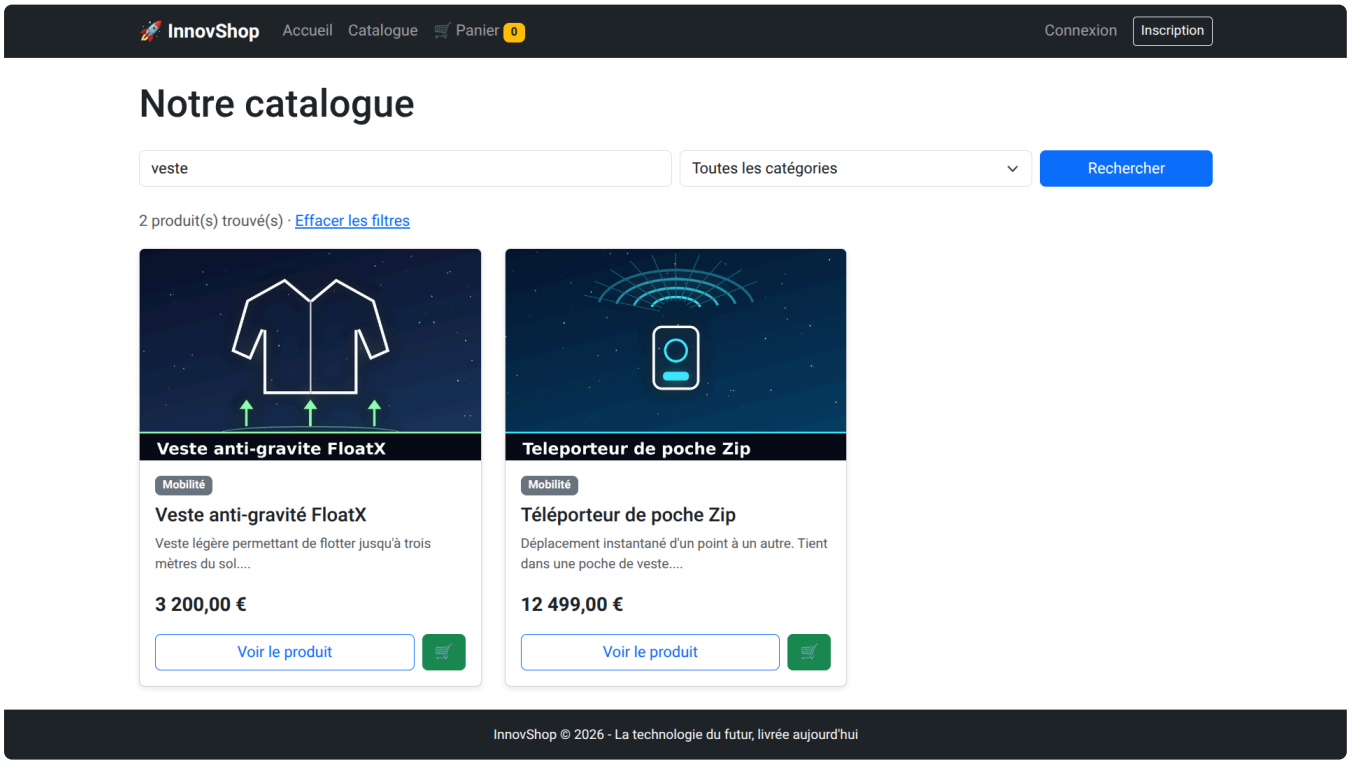
6.3 Une interface responsive

J'ai construit chaque page en « **mobile first** » avec la grille de Bootstrap. Dans le catalogue, chaque carte prend toute la largeur sur téléphone, la moitié sur tablette et un tiers sur ordinateur :

```
{# templates/produit/index.html.twig (extrait) #}
<div class="row g-4">
    {% for produit in produits %}
        <div class="col-12 col-sm-6 col-lg-4">
            {{ include('produit/_carte.html.twig') }}
        </div>
    {% else %}
        <p class="text-muted">Aucun produit ne correspond à votre recherche.
</p>
    {% endfor %}
</div>
```

- `col-12` : 1 carte par ligne sur téléphone ;
- `col-sm-6` : 2 cartes à partir de 576 px de large ;
- `col-lg-4` : 3 cartes à partir de 992 px.

Le menu se replie derrière un **bouton « burger »** sur petit écran (`navbar-toggler`). J'ai testé chaque page avec le mode « appareils » des outils de développement de Chrome.



Le catalogue sur ordinateur : recherche « veste », 2 produits trouvés, 3 colonnes possibles.

6.4 Accessibilité et référencement

- La langue de la page est déclarée (`<html lang="fr">`).
- Chaque image a un **texte alternatif** (`alt`) ; le bouton du menu mobile a un `aria-label` .
- Chaque champ de formulaire a son **label** relié.
- Les images se chargent seulement quand on descend dans la page (`loading="lazy"`).
- Chaque page a une **meta description** et un titre ; un fichier `robots.txt` guide les moteurs de recherche.

Résultat : **100 / 100** en accessibilité et en référencement dans l'audit Lighthouse du site en ligne (partie 10).

6.5 La partie dynamique : le panier en AJAX

Quand le client clique sur « Ajouter au panier » ou « Retirer », la page **ne se recharge pas** : le compteur 🛒 du menu et le total se mettent à jour tout de suite.

```
// assets/app.js (extrait)
document.addEventListener('submit', async (event) => {
    const form = event.target;
    if (!form.classList.contains('js-panier')) {
        return;                       // seulement les formulaires du panier
    }
    event.preventDefault();           // on empêche le rechargement de la
page

    let data;
    try {
        const reponse = await fetch(form.action, {
            method: 'POST',
            body: new FormData(form),  // contient aussi le jeton CSRF
            headers: { 'X-Requested-With': 'XMLHttpRequest' },
        });
        if (!reponse.ok) {
            throw new Error('Refusé par le serveur (code ' + reponse.status +
')');
        }
        data = await reponse.json();
    } catch (erreur) {
        afficherMessage('Action refusée. Rechargez la page et réessayez.',
'danger');
        return;
    }
```

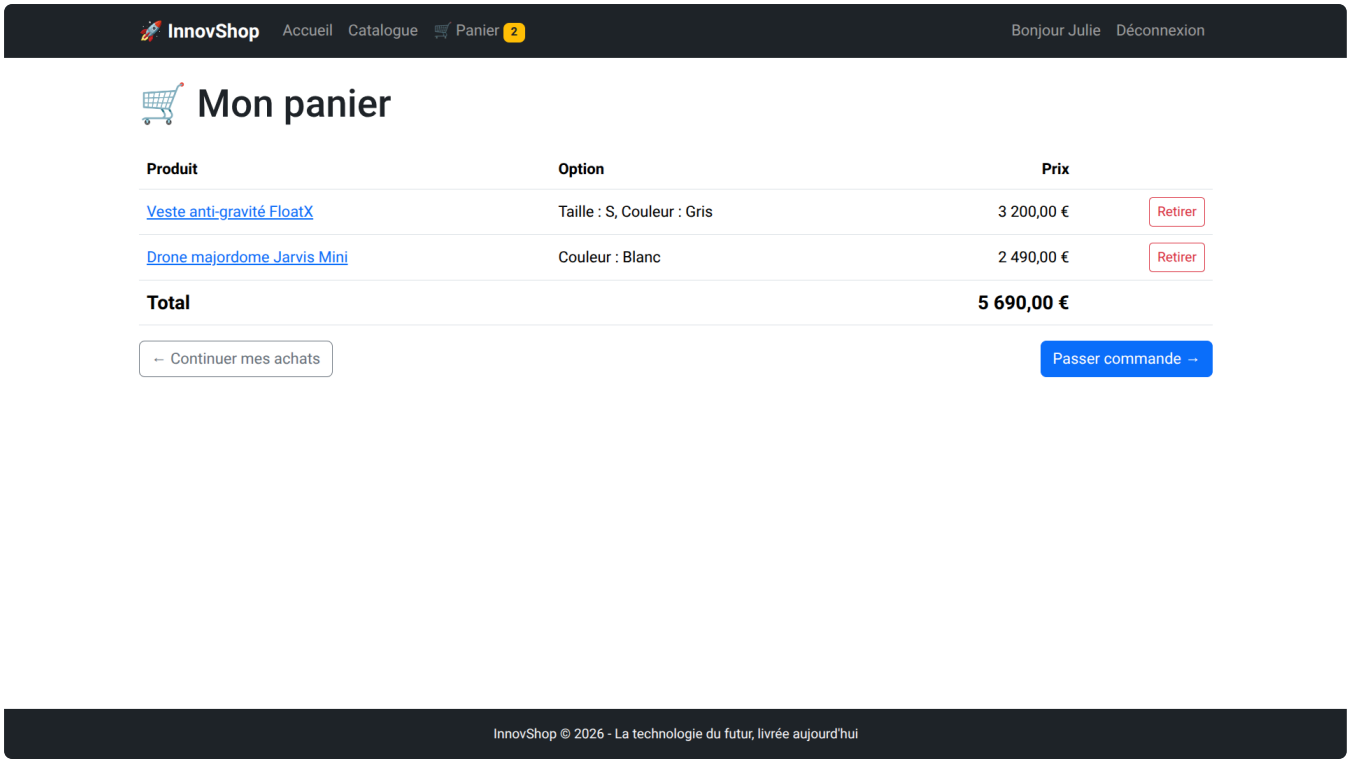
```
    document.getElementById('compteur-panier').textContent = data.nombre;
    afficherMessage(data.message, 'success');
});
```

1. **J'écoute** l'envoi des formulaires qui ont la classe `js-panier`.
2. **J'empêche** le rechargement avec `preventDefault()`.
3. **J'envoie** le formulaire au serveur avec `fetch`. L'en-tête `X-Requested-With` dit au serveur que la demande vient de JavaScript.
4. **Je lis** la réponse JSON (`nombre`, `total`, `message`) et je mets la page à jour.
5. **En cas d'erreur** (jeton refusé, serveur en panne), le `try/catch` affiche un message rouge au lieu de ne rien faire.

Côté serveur, le même contrôleur sert les deux cas : il répond en **JSON** si la demande vient d'AJAX, sinon il redirige vers la page panier. Le site marche donc aussi **sans JavaScript**.

```
// src/Controller/PanierController.php (extrait)
private function repondre(Request $request, PanierService $panier, string
$message): Response
{
    if ($request->isXmlHttpRequest()) {
        return $this->json([
            'message' => $message,
            'nombre'  => $panier->getNombre(),
            'total'   => number_format($panier->getTotal(), 2, ',', ' '),
        ]);
    }
    $this->addFlash('success', $message);

    return $this->redirectToRoute('app_panier');
}
```



Le panier : chaque ajout crée une ligne avec son option. Le compteur 🛒 (2) se met à jour sans recharger la page.

6.6 La sécurité côté navigateur

- **Jeton CSRF** dans chaque formulaire du panier (`<input type="hidden" name="_token" value="{{ csrf_token('panier') }}">`). Le serveur refuse la demande si le jeton est absent ou faux (erreur 400).
- **`textContent` et pas `innerHTML`** pour afficher les messages : un texte qui contient du code s'affiche comme du texte et n'est pas exécuté. C'est une protection contre le **XSS**.
- **Twig échappe automatiquement** tout ce qu'il affiche (`{{ produit.nom }}`).

7. Réalisations back-end

7.1 L'accès aux données avec Doctrine

Les requêtes vers la base sont rangées dans les **Repositories**. J'utilise le **QueryBuilder** de Doctrine : j'écris la requête en PHP, et Doctrine la traduit en SQL.

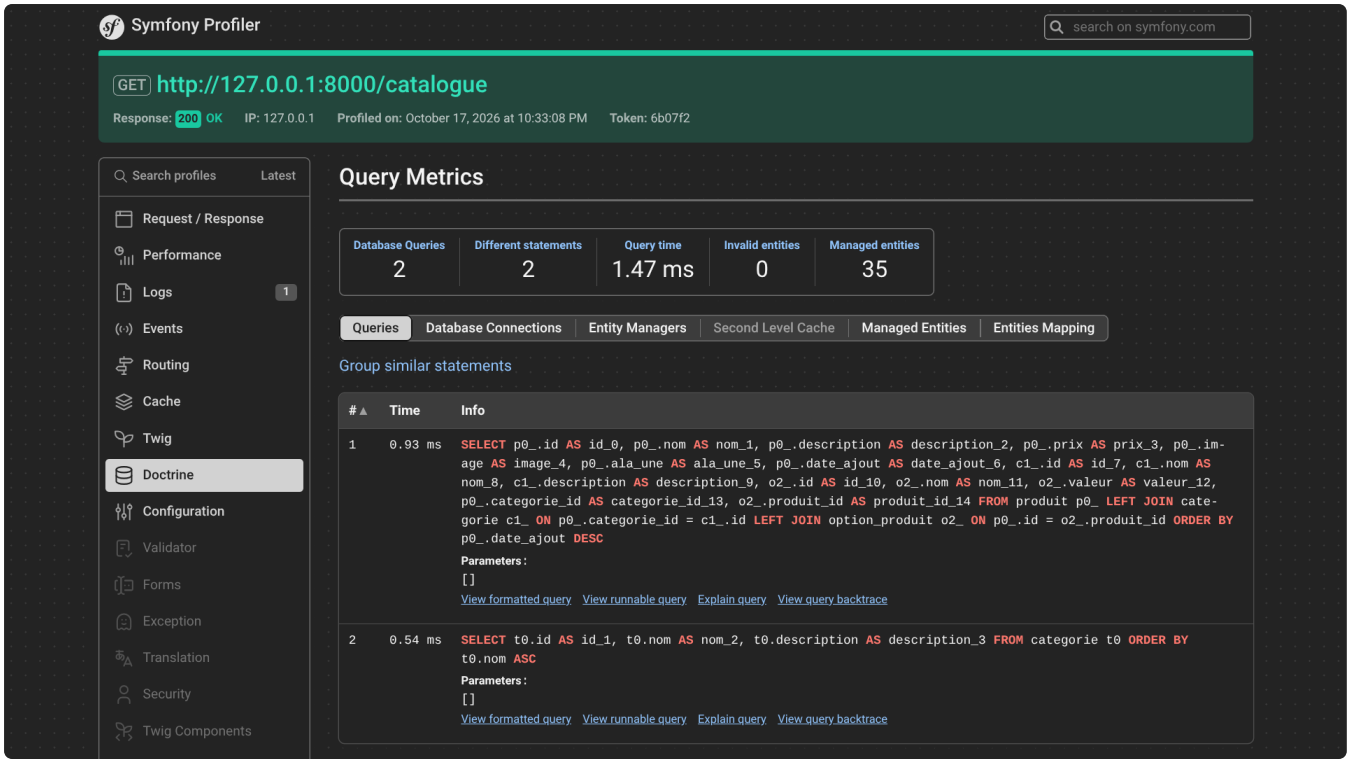
```
// src/Repository/ProduitRepository.php — recherche par mot-clé et par
catégorie
public function rechercher(?string $motCle, ?int $categorieId): array
{
    $qb = $this->createQueryBuilder('p')
        ->leftJoin('p.categorie', 'c')->addSelect('c')
        ->leftJoin('p.options', 'o')->addSelect('o')
        ->orderBy('p.dateAjout', 'DESC');

    if ($motCle) {
        $qb->andWhere('p.nom LIKE :motCle OR p.description LIKE :motCle')
           ->setParameter('motCle', '%' . $motCle . '%');
    }
    if ($categorieId) {
        $qb->andWhere('p.categorie = :categorie')
           ->setParameter('categorie', $categorieId);
    }

    return $qb->getQuery()->getResult();
}
```

- Les deux filtres sont **facultatifs** : la condition n'est ajoutée que si le filtre est rempli.
- **`setParameter()`** : le mot tapé par le visiteur n'est jamais collé dans le SQL. Doctrine envoie une **requête préparée** : c'est la protection contre l'**injection SQL**.
- Côté contrôleur, je nettoie la saisie (`trim`) et je convertis la catégorie en nombre (`(int)`).

L'optimisation : le problème N+1. Avec le Profiler de Symfony, j'ai vu que le catalogue faisait **10 requêtes** : 1 pour la liste des produits, puis 1 de plus pour les options de **chaque** produit affiché. Les deux `leftJoin(...)->addSelect(...)` font charger les catégories et les options **dans la même requête**. Résultat : **2 requêtes** seulement (les produits, puis les catégories du filtre). J'ai aussi ajouté des **index** sur les colonnes de tri (partie 5.3).



Le Profiler de Symfony, onglet Doctrine : 2 requêtes sur le catalogue après la correction. On voit les `LEFT JOIN` dans la première requête.

7.2 Le panier en session (`PanierService`)

Le panier n'est pas une table : il est rangé dans la **session** du visiteur. Un visiteur non connecté peut donc remplir son panier, et la base ne se remplit pas de paniers abandonnés.

```
// src/Service/PanierService.php (extrait)
public function ajouter(int $produitId, ?string $option = null): void
{
    $panier = $this->getSession()->get('panier', []);
    $panier[uniqid()] = ['produitId' => $produitId, 'option' => $option];
    $this->getSession()->set('panier', $panier);
}

public function getTotal(): float
{
    $total = 0;
    foreach ($this->getLignes() as $ligne) {
        $total += (float) $ligne['produit']->getPrix();
    }
    return $total;
}
```

- Chaque ajout crée une **ligne** avec une clé unique (`uniqid()`) : la même veste peut être ajoutée en taille S et en taille M.
- Le panier garde seulement l'**identifiant** du produit. Le prix est relu en base à chaque calcul : le visiteur ne peut pas le modifier.
- Le service est **injecté** par Symfony dans les contrôleurs ; une variable globale Twig affiche le compteur 🛒 sur toutes les pages.

7.3 La validation de la commande (`CommandeController::valider`)

C'est le cœur métier de la boutique. Quand le client clique sur « Je confirme ma commande » :

```
// src/Controller/CommandeController.php (extrait)
#[Route('/valider/{id}', name: 'app_commande_valider', methods: ['POST'])]
#[IsGranted(AdresseVoter::GERER, subject: 'adresse')]
public function valider(Adresse $adresse, Request $request, PanierService
$panier,
                        EntityManagerInterface $em, MailerInterface $mailer):
Response
{
    if (!$this->isCsrfTokenValid('valider-commande', $request->request-
>get('_token'))) {
        throw $this->createAccessDeniedException('Jeton de sécurité
invalide.');
    }

    // 1. La commande, avec un numéro unique
    $commande = new Commande();
    $commande->setNumero('CMD-' . date('Ymd') . '-' .
strtoupper(bin2hex(random_bytes(3))));
    $commande->setStatut('en_attente');
    $commande->setMontantTotal(number_format($panier->getTotal(), 2, '.',
''));
    $commande->setUtilisateur($this->getUser());
    $commande->setAdresse($adresse);

    // 2. Une ligne par article : le nom et le prix sont COPIÉS
    foreach ($panier->getLignes() as $ligne) {
        $ligneCommande = new LigneCommande();
        $ligneCommande->setNomProduit($ligne['produit']->getNom());
        $ligneCommande->setPrixUnitaire($ligne['produit']->getPrix());
        $ligneCommande->setOptionChoisie($ligne['option']);
        $ligneCommande->setProduit($ligne['produit']);
        $commande->addLigne($ligneCommande);
        $em->persist($ligneCommande);
```

```
    }

    // 3. Enregistrement en base, puis on vide le panier
    $em->persist($commande);
    $em->flush();
    $panier->vider();

    // 4. Email de confirmation (la commande reste enregistrée si l'email
échoue)
    // ...
    return $this->redirectToRoute('app_commande_merci', ['numero' =>
$commande->getNumero()]);
}
```

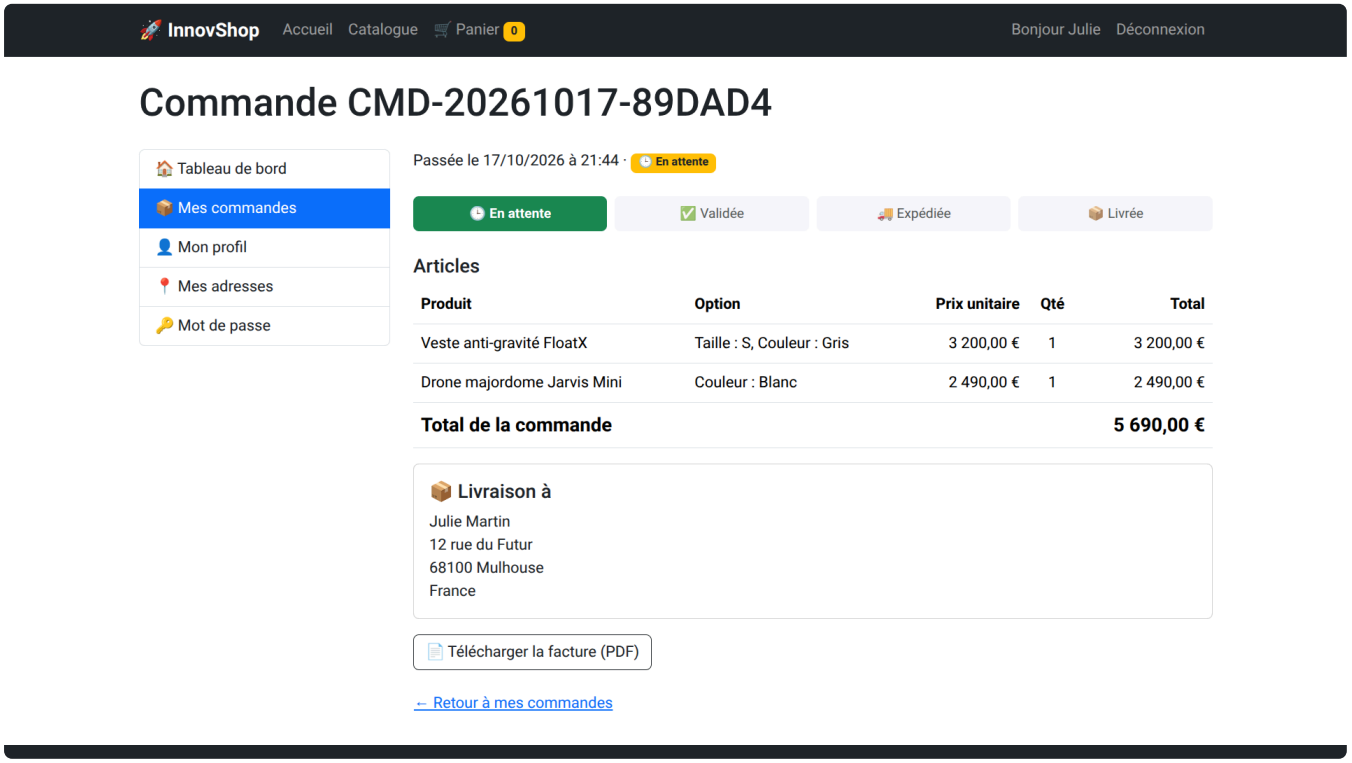
1. **Sécurité d'abord** : la route n'accepte que `POST`, le jeton CSRF est vérifié, et le Voter vérifie que l'adresse appartient bien au client connecté.
2. **Le numéro** combine la date et 6 caractères aléatoires (`random_bytes`) ; la base garantit qu'il est unique.
3. **`persist()` puis `flush()`** : Doctrine prépare les objets, puis enregistre tout en une fois.
4. **L'email** est envoyé avec Symfony Mailer et un template Twig. S'il échoue, un message prévient le client mais la commande est gardée.

7.4 Les règles d'accès : les Voters

Un client connecté ne doit pas pouvoir ouvrir la commande d'un autre client en changeant le numéro dans l'adresse de la page (faille appelée **IDOR**). Le `CommandeVoter` répond à la question « a-t-il le droit de voir cette commande ? » :

```
// src/Security/Voter/CommandeVoter.php (extrait)
protected function voteOnAttribute(string $attribute, mixed $subject,
TokenInterface $token): bool
{
    $utilisateur = $token->getUser();
    if (!$utilisateur instanceof Utilisateur) {
        return false;                                   // pas connecté :
refusé
    }
    if ($this->security->isGranted('ROLE_ADMIN')) {
        return true;                                    // un admin peut tout
voir
    }
    return $subject->getUtilisateur()?->getId() === $utilisateur->getId();
// le propriétaire
}
```

Il s'utilise avec un simple attribut au-dessus de l'action : `#[IsGranted(CommandeVoter::VOIR, subject: 'commande')]`. Le même principe protège les adresses (`AdresseVoter`).



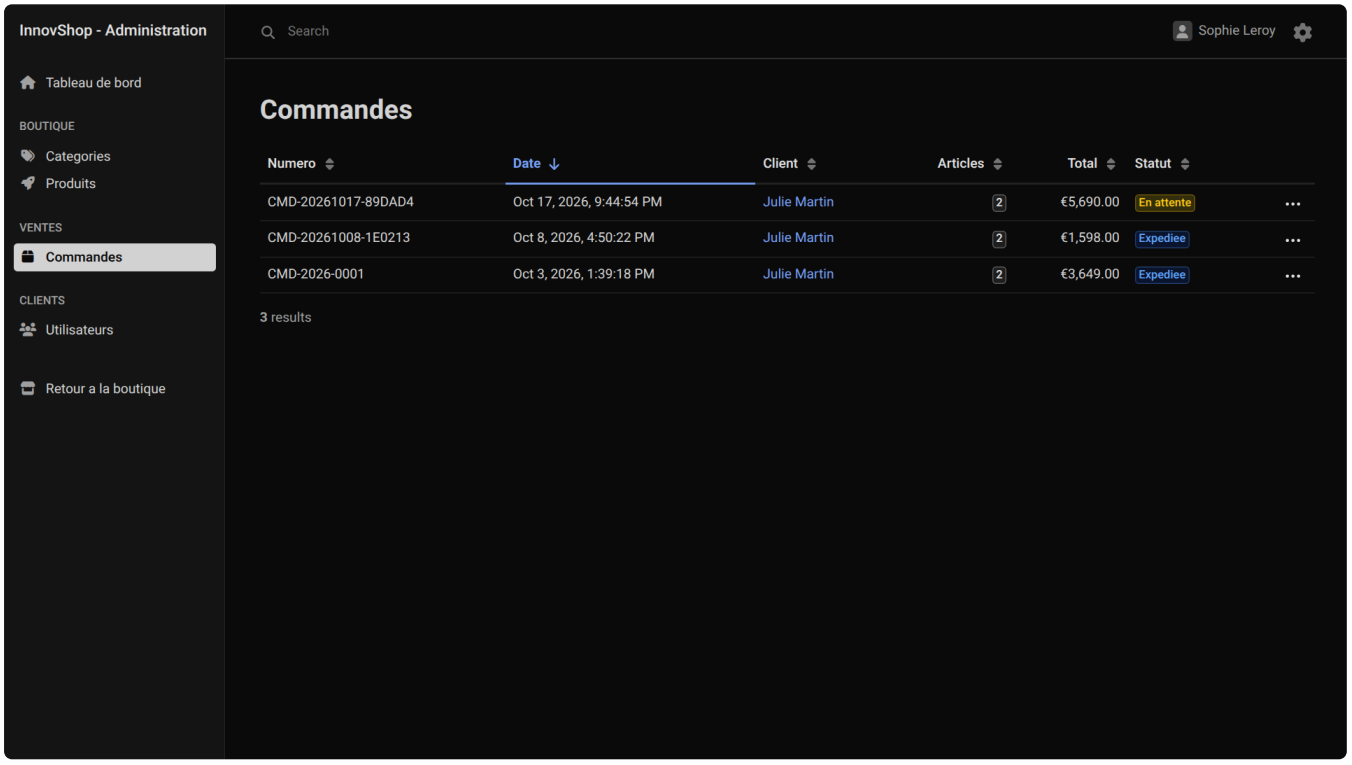
L'espace client : le détail d'une commande, le suivi de son statut et le bouton de la facture. Seul son propriétaire peut ouvrir cette page.

7.5 L'email automatique au changement de statut

Quand l'administrateur passe une commande de « validée » à « expédiée » dans le back-office, le client reçoit un email. J'ai utilisé un **écouteur d'événements Doctrine** (`CommandeStatutListener`) en deux temps :

- **`preUpdate`** (juste avant l'enregistrement) : il regarde si le champ `statut` a changé (`hasChangedField('statut')`) ;
- **`postUpdate`** (juste après) : il envoie l'email, seulement si l'enregistrement a réussi.

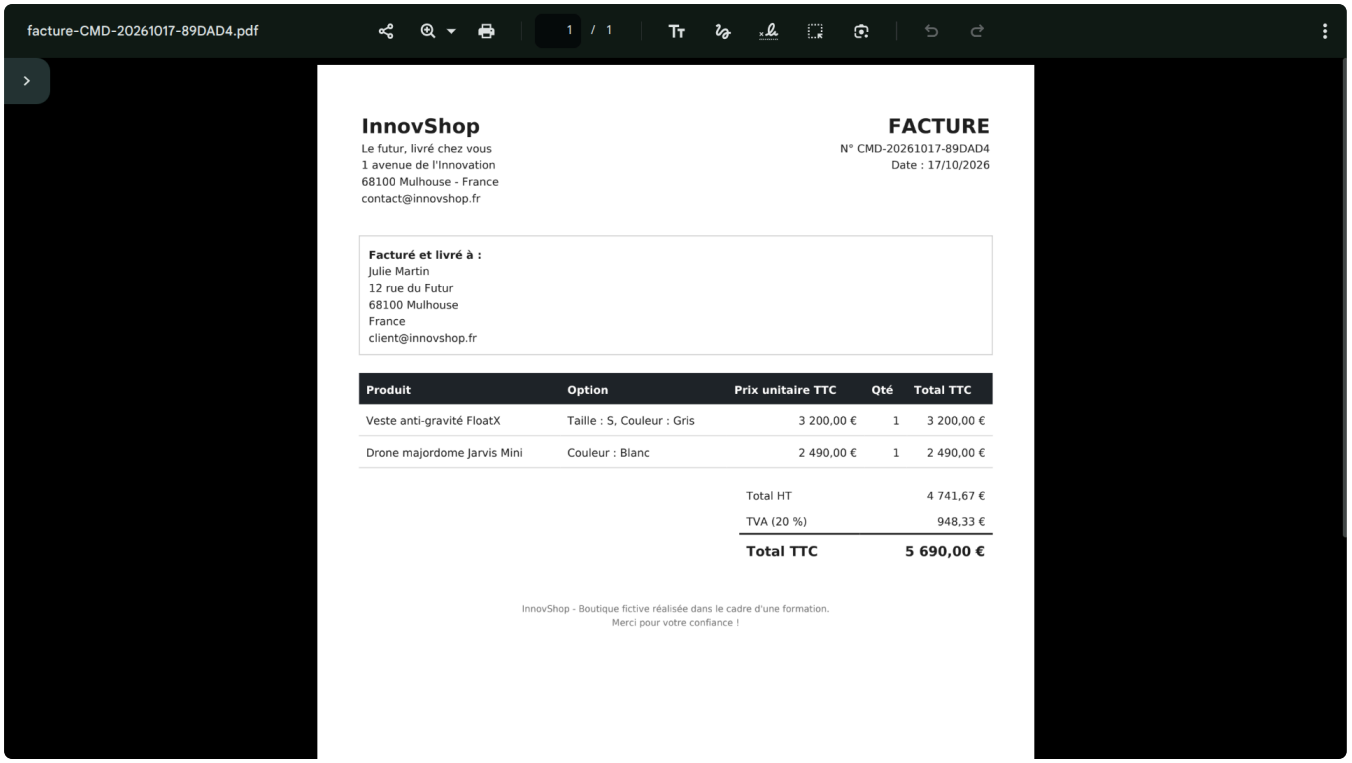
Avantage : je n'ai pas modifié le back-office EasyAdmin, et l'email part quel que soit l'endroit où le statut est changé.



Le back-office EasyAdmin : les commandes et leur statut.

7.6 La facture PDF (`FactureService`)

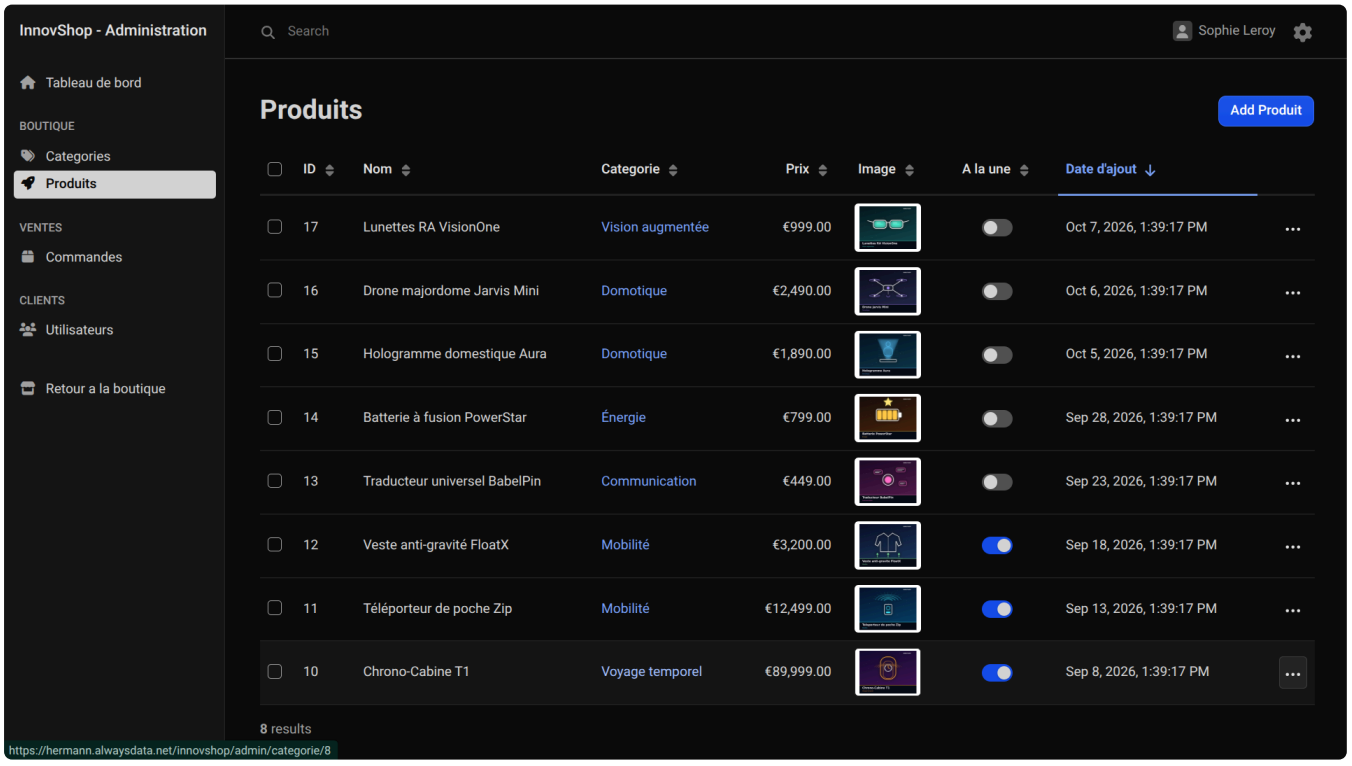
Le service fabrique le HTML de la facture avec un template Twig, puis **Dompdf** le transforme en PDF A4. La police DejaVu Sans affiche correctement les accents et le symbole €. La facture est protégée par le même Voter que le détail de la commande, et elle n'existe pas pour une commande annulée.



La facture générée : total HT, TVA 20 % et total TTC.

7.7 Le back-office EasyAdmin

L'administrateur gère les catégories, les produits (avec téléversement d'images et case « à la une »), les commandes et les utilisateurs. Tout `/admin` est réservé au rôle `ROLE_ADMIN`.



Le back-office : la liste des produits, avec image, catégorie, prix et interrupteur « à la une ».

7.8 La sécurité côté serveur

Risque	Protection dans InnovShop
Mot de passe volé en base	Hachage **bcrypt** avec le password hasher de Symfony
Pages réservées	`access_control` : `/admin` pour `ROLE_ADMIN`, `/compte` et `/commande` pour `ROLE_USER` ; un admin a aussi les droits d'un client (`role_hierarchy`)
Accès aux données d'un autre client	Voters `CommandeVoter` et `AdresseVoter`
Formulaire envoyé depuis un autre site	Jetons **CSRF** sur tous les formulaires
Données invalides	Contraintes `Assert` vérifiées côté serveur (code postal à 5 chiffres, mot de passe de 8 caractères avec une majuscule et un chiffre…)
Fuite d'informations	Pages d'erreur 403, 404 et 500 personnalisées ; secrets uniquement dans `.env.local`

8. Jeu d'essai : passer une commande

J'ai choisi la fonctionnalité la plus représentative de la boutique : **passer une commande**. Elle traverse tout le projet : panier en session, authentification, formulaire validé, Voter, base de données, email.

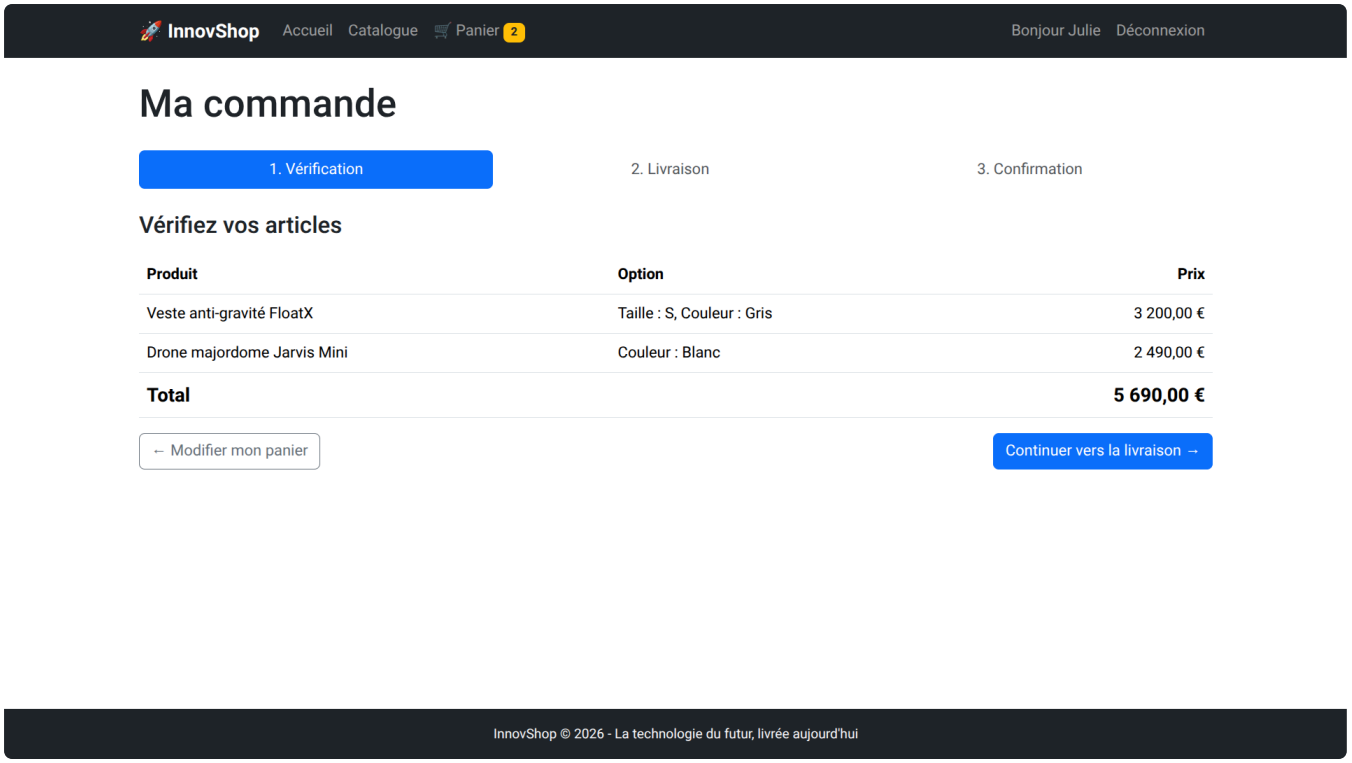
8.1 Les données de test

- **Cliente :** Julie Martin, compte client de démonstration.
- **Panier :** Veste anti-gravité FloatX (taille S, couleur Gris) à 3 200,00 € + Drone majordome Jarvis Mini (couleur Blanc) à 2 490,00 €.
- **Adresse :** 12 rue du Futur, 68100 Mulhouse, France.

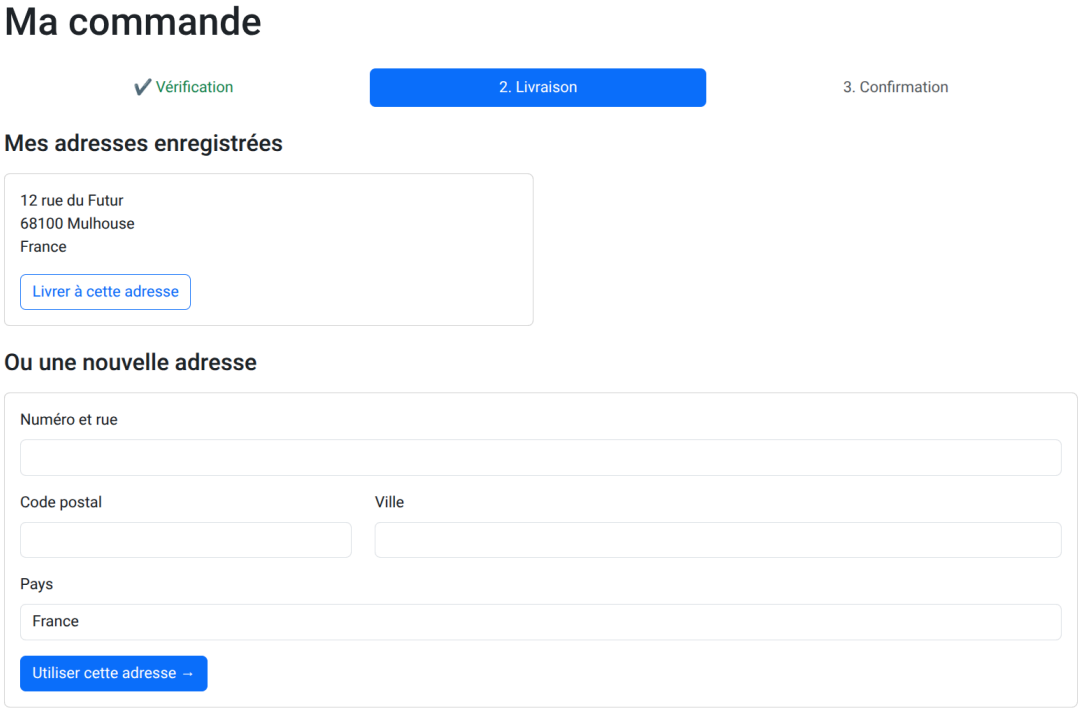
8.2 Le scénario normal

Étape	Action	Résultat attendu	Résultat obtenu
1	Ajouter les 2 produits au panier	Le compteur 🛒 passe à 2 sans recharger la page	Conforme

Étape	Action	Résultat attendu	Résultat obtenu
2	Ouvrir le panier	2 lignes avec leurs options, total 5 690,00 €	Conforme
3	Étape 1 — Vérification	Le récapitulatif affiche les 2 articles et le total	Conforme
4	Étape 2 — Livraison	Choisir l'adresse enregistrée de Julie	Conforme
5	Étape 3 — Confirmation, puis « Je confirme ma commande »	Page « Merci » avec un numéro au format `CMD-AAAAMMJJ-XXXXXX`	Conforme : CMD-20261017-89DAD4
6	Regarder le compteur du panier	Le panier est vidé (0)	Conforme
7	Ouvrir la boîte Mailtrap	Email « Confirmation de votre commande CMD-20261017-89DAD4 » reçu	Conforme
8	Espace client → Mes commandes	La commande apparaît avec le statut « En attente »	Conforme
9	Télécharger la facture	PDF : total HT 4 741,67 €, TVA 948,33 €, total TTC 5 690,00 €	Conforme
10	Admin : passer le statut à « Expédiée »	Un email de changement de statut est envoyé à Julie	Conforme



Étape 1 : le client vérifie ses articles et le total avant de continuer.



Étape 2 : il choisit une adresse enregistrée ou en saisit une nouvelle.

Image — Commande étape 3 : confirmation

Étape 3 : le récapitulatif final et le bouton « Je confirme ma commande ».

8.3 Les cas d'erreur

Cas testé	Résultat attendu	Résultat obtenu
Un visiteur non connecté ouvre `/commande/verification`	Redirection vers la page de connexion	Conforme
Un client ouvre la commande avec un panier vide	Retour au panier avec le message « Votre panier est vide. »	Conforme
Code postal « 6810 » dans le formulaire d'adresse	Message rouge « Le code postal doit contenir 5 chiffres. », rien n'est enregistré	Conforme
Un client met l'identifiant de l'adresse d'un autre client dans l'URL	Accès refusé (403) par l'`AdresseVoter`	Conforme
Jeton CSRF modifié dans le formulaire du panier	Requête refusée (400) et message rouge « Action refusée »	Conforme
Un client ouvre la facture d'une commande qui n'est pas la sienne	Accès refusé (403) par le `CommandeVoter`	Conforme

Une anomalie trouvée grâce à ce jeu d'essai : au début, un jeton CSRF modifié dans le panier échouait en silence : rien ne se passait à l'écran. J'ai corrigé de deux côtés : le serveur renvoie maintenant une erreur 400, et le JavaScript l'attrape (`try/catch`) pour afficher un message à l'utilisateur.

8.4 Les tests automatisés (PHPUnit)

En plus des tests à la main, j'ai écrit **17 tests PHPUnit** qui se relancent en une commande, sur une base de test séparée (`innovshop_test`).

Type	Fichier	Ce qui est vérifié
Unitaire	`tests/Entity/ProduitTest.php`	Valeurs par défaut d'un produit, regroupement des options par nom
Unitaire	`tests/Entity/CommandeTest.php`	Regroupement des lignes identiques pour compter les quantités

Type	Fichier	Ce qui est vérifié
Fonctionnel	`tests/Controller/PagesPubliquesTest.php`	Accueil, catalogue, recherche, produit inexistant (404), ajout au panier refusé en GET (405)
Fonctionnel	`tests/Controller/SecuriteAccesTest.php`	Redirection vers la connexion, administration interdite au client (403), un client voit sa commande, le Voter bloque la commande d'un autre (403)

```
// tests/Controller/SecuriteAccesTest.php (extrait)
public function testEspaceClientRedirigeVersConnexionSiAnonyme(): void
{
    $client = static::createClient();
    $client->request('GET', '/compte');

    $this->assertResponseRedirects('/login');
}
```

Image — Résultat PHPUnit

`php vendor/bin/phpunit` *: OK (17 tests, 35 assertions).*

9. Veille sur les vulnérabilités de sécurité

9.1 Ma référence : l'OWASP Top 10

L'**OWASP** (Open Worldwide Application Security Project) est une fondation qui publie le **Top 10** des risques de sécurité les plus importants pour les applications web. Pour ce projet, j'ai étudié la **version 2025** et j'ai vérifié, risque par risque, comment InnovShop se protège.

Risque OWASP 2025	En clair	Protection dans InnovShop
A01 — Broken Access Control	Quelqu'un accède à ce qui ne lui appartient pas	`access_control` par rôle, `#[IsGranted]`, Voters sur les commandes et les adresses (protection contre l'IDOR)
A02 — Security Misconfiguration	Un réglage oublié ouvre une porte	En production : `APP_ENV=prod`, `APP_DEBUG=0`, Profiler désactivé, seul le dossier `public/` est visible depuis Internet, pages d'erreur personnalisées
A03 — Software Supply Chain Failures	Une bibliothèque utilisée est vulnérable	Bibliothèques connues et maintenues (Symfony LTS, Doctrine, EasyAdmin) ; versions fixées dans `composer.lock` ; `--no-dev` en production
A04 — Cryptographic Failures	Des données sensibles mal protégées	Mots de passe hachés en **bcrypt** ; site en **HTTPS** ; secrets dans `.env.local`, jamais sur GitHub
A05 — Injection	Du code malveillant caché dans une saisie (SQL, XSS)	Requêtes préparées avec `setParameter()` ; échappement automatique de Twig ; `textContent` en JavaScript
A06 — Insecure Design	Une faille dans la logique même	Le prix est toujours relu en base (jamais envoyé par le navigateur) ; nom et prix copiés dans la commande
A07 — Authentication Failures	Connexion ou mots de passe faibles	Système de sécurité de Symfony ; mot de passe de 8 caractères minimum avec une majuscule et un chiffre
A08 — Software or Data Integrity Failures	Des données modifiées sans contrôle	Jetons **CSRF** sur tous les formulaires ; validation `Assert` côté serveur

Risque OWASP 2025	En clair	Protection dans InnovShop
A09 — Security Logging and Alerting Failures	On ne voit pas qu'une attaque a lieu	Journaux d'erreurs de Symfony ; piste d'amélioration (partie 11)
A10 — Mishandling of Exceptional Conditions	Une erreur mal gérée laisse fuir des informations	`try/catch` autour de l'envoi des emails ; pages d'erreur 403, 404 et 500 qui ne montrent aucun détail technique

9.2 Trois failles que j'ai traitées en détail

L'IDOR (A01). Les commandes s'ouvrent par leur numéro dans l'adresse de la page. Sans protection, un client pourrait essayer d'autres numéros. Le `CommandeVoter` vérifie à chaque fois que la commande appartient au client connecté ; sinon, erreur 403. Un test PHPUnit vérifie ce refus automatiquement.

Le CSRF (A08). Un site pirate pourrait cacher un formulaire qui ajoute des articles ou valide une commande à la place du client. Chaque formulaire contient un jeton secret, différent pour chaque session ; le serveur refuse toute demande sans le bon jeton.

Le XSS (A05). Si un pirate arrivait à enregistrer du code JavaScript dans un nom de produit, il pourrait s'exécuter chez les visiteurs. Twig transforme les caractères spéciaux (`<` devient `<`) et mon JavaScript n'utilise que `textContent` : le code s'affiche comme du texte, il n'est jamais exécuté.

9.3 Comment je fais ma veille

Source	Ce que j'y trouve
OWASP Top 10	Les grandes familles de risques et les bonnes pratiques pour s'en protéger
Blog Symfony — Security Advisories	Les failles découvertes dans Symfony et les versions qui les corrigent
CERT-FR	Les alertes de sécurité officielles en France
Commande `composer audit`	Liste les bibliothèques du projet qui ont une faille connue

La documentation de l'OWASP est en anglais : cette veille me fait aussi pratiquer l'anglais technique, demandé dans l'épreuve du questionnaire professionnel.

10. Déploiement

InnovShop est en ligne chez l'hébergeur français **Alwaysdata** : https://hermann.alwaysdata.net/innovshop/. J'ai écrit toute la procédure dans le **README** du dépôt GitHub, pour qu'une autre personne puisse la refaire.

10.1 La mise en ligne, étape par étape

1. **Préparer l'hébergement** : une base MariaDB 11.4 et un site PHP 8.4. Le **répertoire racine pointe sur `public/`** : seul ce dossier est visible depuis Internet, le code et les secrets restent protégés.
2. **Récupérer le code** en SSH avec `git clone`.
3. **Configurer la production** dans un fichier `.env.local` créé sur le serveur (jamais sur GitHub) : `APP_ENV=prod`, `APP_DEBUG=0`, `APP_SECRET`, `DATABASE_URL`, `MAILER_DSN`, `MAILER_FROM`.
4. **Installer et construire** :

```
composer install --no-dev --optimize-autoloader      # seulement les bibliothèques de production
php bin/console doctrine:migrations:migrate --no-interaction   # crée les tables
php bin/console minify:install
php bin/console asset-map:compile                    # CSS et JS minifiés et versionnés
php bin/console cache:clear
```

5. **Les emails** partent vers une boîte de test **Mailtrap** (SMTP), car les comptes de démonstration ont des adresses fictives.
6. **Vérifier** tout le parcours en ligne (catalogue, panier, commande, facture, back-office) et les pages d'erreur.

10.2 Les problèmes rencontrés et résolus

Problème	Cause	Solution
Sous-domaine refusé	Le domaine `alwaysdata.net` n'appartient pas au compte	Site publié dans un chemin : `hermann.alwaysdata.net/innovshop`
Connexion à la base impossible	Le mot de passe contenait un `@`, qui coupe l'adresse de connexion	Caractère encodé en `%40` dans `DATABASE_URL`

Problème	Cause	Solution
Erreur « Not Found »	Répertoire racine mal renseigné	Racine corrigée vers `/innovshop/public/`
Emails non envoyés	Mauvais type de connexion Mailtrap	Passage à la connexion SMTP de la boîte de test

10.3 La mise à jour du site

```
cd ~/innovshop
git pull
composer install --no-dev --optimize-autoloader
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console asset-map:compile
php bin/console cache:clear
```

10.4 Le résultat : Lighthouse 100 / 100 / 100 / 100

J'ai mesuré la qualité du site en ligne avec l'audit **Lighthouse** de Chrome, en navigation privée. Le premier audit, en local, donnait 98 / 96 / 100 / 54. Après les corrections de la séquence 9 (meta description, `robots.txt`, fichiers minifiés, images en chargement différé) et le passage en mode production, le site en ligne obtient 100 partout.

Image — Lighthouse 100 / 100 / 100 / 100

Lighthouse sur https://hermann.alwaysdata.net/innovshop/ : 100 en performances, accessibilité, bonnes pratiques et SEO.

11. Bilan et conclusion

11.1 Ce qui a été réalisé

- **41 user stories sur 42** terminées ; la dernière a été écartée volontairement, avec une raison écrite.
- Un site **complet et en ligne** : vitrine, panier, commande, espace client, back-office.
- **17 tests automatisés** au vert et un jeu d'essai conforme.
- **Lighthouse 100** dans les 4 catégories, code conforme à la norme **PSR-12**.

11.2 Les difficultés et ce qu'elles m'ont appris

Difficulté	Ce que j'ai appris
Le catalogue faisait 10 requêtes (problème N+1)	Mesurer avant d'optimiser : le Profiler m'a montré la vraie cause (les options), et une jointure l'a réglée
La recherche ne répondait pas	La navigation Turbo de Symfony interceptait le formulaire ; je l'ai désactivée sur ce formulaire (`data-turbo="false"`)
Un jeton CSRF falsifié échouait sans rien afficher	Toujours prévoir le cas d'erreur, côté serveur (code 400) **et** côté navigateur (`try/catch`)
Formulaire vide qui plantait (`null` au lieu d'un texte)	L'option `empty_data` des formulaires Symfony
La mise en ligne (sous-domaine, `@` dans le mot de passe, répertoire racine)	Lire les messages d'erreur un par un, et documenter chaque solution dans le README

11.3 Ce dont je suis le plus satisfait

- La **commande en 3 étapes**, sécurisée de bout en bout (rôle, Voter, CSRF, validation).
- L'**email automatique** au changement de statut, sans toucher au back-office, grâce à un écouteur Doctrine.
- Le passage de **10 à 2 requêtes** sur le catalogue, et le score **Lighthouse 100**.

11.4 Les améliorations possibles

- Ajouter un **paiement en ligne** (par exemple Stripe en mode test).
- Gérer les **quantités** dans le panier (aujourd'hui, chaque ajout crée une ligne) et le **stock** des produits.
- Renforcer la **journalisation** des actions sensibles (connexions ratées, changements de statut).
- Lancer `composer audit` et les **tests** automatiquement à chaque envoi sur GitHub (intégration continue).

11.5 Conclusion

InnovShop m'a fait réaliser une application web complète, de l'analyse du besoin jusqu'à la mise en ligne. J'ai mis en œuvre les 8 compétences du titre dans un même projet, avec un framework professionnel et les bonnes pratiques de sécurité. C'est le projet qui m'a le

plus fait progresser, et il me donne une base solide pour mes premiers projets de développeur web.

Annexes : la fonctionnalité « passer une commande »

Ces annexes contiennent le code complet de la fonctionnalité la plus représentative, côté back-end et côté front-end. Le reste du code est sur GitHub : https://github.com/LettreH/innovshop

A.1 Le contrôleur de la commande (back-end)

```
// src/Controller/CommandeController.php
#[Route('/commande')]
#[IsGranted('ROLE_USER')]
final class CommandeController extends AbstractController
{
    // ===== ÉTAPE 1 : vérification du panier =====
    #[Route('/verification', name: 'app_commande_verification')]
    public function verification(PanierService $panier): Response
    {
        if ($panier->getNombre() === 0) {
            $this->addFlash('warning', 'Votre panier est vide.');
            return $this->redirectToRoute('app_panier');
        }

        return $this->render('commande/verification.html.twig', [
            'lignes' => $panier->getLignes(),
            'total'  => $panier->getTotal(),
        ]);
    }

    // ===== ÉTAPE 2 : adresse de livraison =====
    #[Route('/livraison', name: 'app_commande_livraison')]
    public function livraison(Request $request, PanierService $panier,
EntityManagerInterface $em): Response
    {
        if ($panier->getNombre() === 0) {
            return $this->redirectToRoute('app_panier');
        }

        $adresse = new Adresse();
        $adresse->setPays('France');

        $form = $this->createForm(AdresseType::class, $adresse);
        $form->handleRequest($request);
```

```
        if ($form->isSubmitted() && $form->isValid()) {
            $adresse->setUtilisateur($this->getUser());
            $em->persist($adresse);
            $em->flush();

            return $this->redirectToRoute('app_commande_confirmation', ['id'
=> $adresse->getId()]);
        }

        return $this->render('commande/livraison.html.twig', [
            'form'     => $form,
            'adresses' => $this->getUser()->getAdresses(),
        ]);
    }

    // ===== ÉTAPE 3 : récapitulatif avant validation =====
    #[Route('/confirmation/{id}', name: 'app_commande_confirmation',
requirements: ['id' => '\d+'])]
    #[IsGranted(AdresseVoter::GERER, subject: 'adresse')]
    public function confirmation(Adresse $adresse, PanierService $panier):
Response
    {
        if ($panier->getNombre() === 0) {
            return $this->redirectToRoute('app_panier');
        }

        return $this->render('commande/confirmation.html.twig', [
            'lignes'  => $panier->getLignes(),
            'total'   => $panier->getTotal(),
            'adresse' => $adresse,
        ]);
    }

    // ===== VALIDATION : enregistrement en base + email =====
    #[Route('/valider/{id}', name: 'app_commande_valider', methods: ['POST'],
requirements: ['id' => '\d+'])]
    #[IsGranted(AdresseVoter::GERER, subject: 'adresse')]
    public function valider(
        Adresse $adresse,
        Request $request,
        PanierService $panier,
        EntityManagerInterface $em,
        MailerInterface $mailer
    ): Response {
        if (!$this->isCsrfTokenValid('valider-commande', $request->request-
```

```
>get('_token'))) {
            throw $this->createAccessDeniedException('Jeton de sécurité
invalide.');
        }

        $lignes = $panier->getLignes();
        if (!$lignes) {
            return $this->redirectToRoute('app_panier');
        }

        // 1. La commande
        $commande = new Commande();
        $commande->setNumero('CMD-' . date('Ymd') . '-' .
strtoupper(bin2hex(random_bytes(3))));
        $commande->setDateCommande(new \DateTimeImmutable());
        $commande->setStatut('en_attente');
        $commande->setMontantTotal(number_format($panier->getTotal(), 2, '.',
''));
        $commande->setUtilisateur($this->getUser());
        $commande->setAdresse($adresse);

        // 2. Une ligne de commande par article du panier
        foreach ($lignes as $ligne) {
            $produit = $ligne['produit'];

            $ligneCommande = new LigneCommande();
            $ligneCommande->setNomProduit($produit->getNom());
            $ligneCommande->setPrixUnitaire($produit->getPrix());
            $ligneCommande->setOptionChoisie($ligne['option']);
            $ligneCommande->setProduit($produit);

            $commande->addLigne($ligneCommande);
            $em->persist($ligneCommande);
        }

        // 3. Enregistrement en base, puis on vide le panier
        $em->persist($commande);
        $em->flush();
        $panier->vider();

        // 4. Email de confirmation
        $email = (new TemplatedEmail())
            ->to($this->getUser()->getEmail())
            ->subject('Confirmation de votre commande ' . $commande-
>getNumero())
            ->htmlTemplate('emails/confirmation_commande.html.twig')
```

```
            ->context(['commande' => $commande]);

        try {
            $mailer->send($email);
        } catch (TransportExceptionInterface $e) {
            // La commande est enregistrée même si l'email échoue
            $this->addFlash('warning', 'Votre commande est enregistrée, mais
l\'email n\'a pas pu être envoyé.');
        }

        return $this->redirectToRoute('app_commande_merci', ['numero' =>
$commande->getNumero()]);
    }

    // ===== PAGE DE REMERCIEMENT =====
    #[Route('/merci/{numero}', name: 'app_commande_merci')]
    public function merci(string $numero, CommandeRepository
$commandeRepository): Response
    {
        $commande = $commandeRepository->findOneBy([
            'numero'      => $numero,
            'utilisateur' => $this->getUser(),
        ]);

        if (!$commande) {
            throw $this->createNotFoundException('Commande introuvable.');
        }

        return $this->render('commande/merci.html.twig', [
            'commande' => $commande,
        ]);
    }
}
```

A.2 Le contrôleur du panier (back-end)

```
// src/Controller/PanierController.php
#[Route('/panier')]
final class PanierController extends AbstractController
{
    // Afficher le panier
    #[Route('', name: 'app_panier')]
    public function index(PanierService $panier): Response
    {
        return $this->render('panier/index.html.twig', [
            'lignes' => $panier->getLignes(),
```

```
            'total'  => $panier->getTotal(),
        ]);
    }

    // Ajouter un produit (avec ses options éventuelles)
    #[Route('/ajouter/{id}', name: 'app_panier_ajouter', methods: ['POST'],
requirements: ['id' => '\d+'])]
    public function ajouter(Produit $produit, Request $request, PanierService
$panier): Response
    {
        if (!$this->isCsrfTokenValid('panier', $request->request-
>get('_token'))) {
            throw new BadRequestHttpException('Jeton de sécurité invalide.');
        }

        // Les options arrivent sous la forme : ['Taille' => 'M', 'Couleur'
=> 'Noir']
        $choix = [];
        foreach ($request->request->all('options') as $nom => $valeur) {
            $choix[] = $nom . ' : ' . $valeur;
        }
        $option = $choix ? mb_substr(implode(', ', $choix), 0, 150) : null;

        $panier->ajouter($produit->getId(), $option);

        return $this->repondre($request, $panier, $produit->getNom() . ' a
été ajouté au panier.');
    }

    // Supprimer une ligne du panier
    #[Route('/supprimer/{cle}', name: 'app_panier_supprimer', methods:
['POST'])]
    public function supprimer(string $cle, Request $request, PanierService
$panier): Response
    {
        if (!$this->isCsrfTokenValid('panier', $request->request-
>get('_token'))) {
            throw new BadRequestHttpException('Jeton de sécurité invalide.');
        }

        $panier->supprimer($cle);

        return $this->repondre($request, $panier, 'Article retiré du
panier.');
    }
```

```
    // Répond en JSON si la demande vient du JavaScript (AJAX), sinon
redirige vers la page panier.
    private function repondre(Request $request, PanierService $panier, string
$message): Response
    {
        if ($request->isXmlHttpRequest()) {
            return $this->json([
                'message' => $message,
                'nombre'  => $panier->getNombre(),
                'total'   => number_format($panier->getTotal(), 2, ',', ' '),
            ]);
        }

        $this->addFlash('success', $message);

        return $this->redirectToRoute('app_panier');
    }
}
```

A.3 La page de confirmation (front-end)

```
{# templates/commande/confirmation.html.twig #}
{% extends 'base.html.twig' %}

{% block title %}Confirmation - InnovShop{% endblock %}

{% block body %}
    <h1 class="mb-4">Ma commande</h1>
    {{ include('commande/_etapes.html.twig', {etape: 3}) }}

    <div class="row g-4">
        <div class="col-12 col-lg-8">
            <h2 class="h4 mb-3">Récapitulatif</h2>
            {{ include('commande/_recapitulatif.html.twig') }}
        </div>

        <div class="col-12 col-lg-4">
            <div class="card">
                <div class="card-body">
                    <h2 class="h5">📦 Livraison à</h2>
                    <p>
                        {{ app.user.prenom }} {{ app.user.nom }}<br>
                        {{ adresse.rue }}<br>
                        {{ adresse.codePostal }} {{ adresse.ville }}<br>
                        {{ adresse.pays }}
                    </p>
```

```
                    <form method="post" action="{{
path('app_commande_valider', {id: adresse.id}) }}" data-turbo="false">
                        <input type="hidden" name="_token" value="{{
csrf_token('valider-commande') }}">
                        <button type="submit" class="btn btn-success btn-lg
w-100">✅ Je confirme ma commande</button>
                    </form>
                </div>
            </div>
            <a href="{{ path('app_commande_livraison') }}" class="btn btn-
link px-0 mt-3">← Changer d'adresse</a>
        </div>
    </div>
{% endblock %}
```

A.4 La barre des étapes (front-end)

```
{# templates/commande/_etapes.html.twig #}
<ul class="nav nav-pills nav-fill mb-4 gap-2">
    {% for numero, nom in {1: 'Vérification', 2: 'Livraison', 3:
'Confirmation'} %}
        <li class="nav-item">
            <span class="nav-link {{ numero == etape ? 'active' : (numero <
etape ? 'text-success' : 'text-muted') }}">
                {% if numero < etape %}✔{% else %}{{ numero }}.{% endif %} {{
nom }}
            </span>
        </li>
    {% endfor %}
</ul>
```

Le service du panier (`PanierService`) et le JavaScript du panier (`assets/app.js`) sont présentés dans les parties 6.5 et 7.2.